



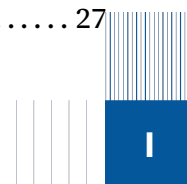
UNIVERSITÉ
LIBRE
DE BRUXELLES

Digital Twin - Runbook

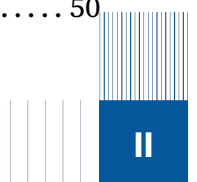


Contents

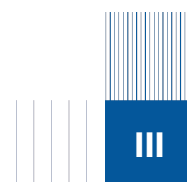
1 Overview	1
1.1 Architecture overview	1
1.1.1 Services and responsibilities	1
1.1.1.1 Collector service	1
1.1.1.2 Preprocessing service	1
1.1.1.3 Database service	1
1.1.1.4 Alert engine	1
1.1.1.5 Web application service	2
1.1.2 Control service (future)	2
1.1.3 Main data flows (high level)	3
1.2 Conventions (time, units, naming)	5
1.2.1 Time representation	5
1.2.2 Units	5
2 Quickstart	5
2.1 Prerequisites	5
2.1.1 Hardware	5
2.1.1.1 Wiring Details	7
2.1.2 Software	11
2.2 Installation guidelines	12
2.2.1 Poetry profiles	12
2.2.2 Setup Kafka (KRaft)	12
2.2.3 Setup PostgreSQL/TimescaleDB	14
2.2.3.1 Database Migrations	14
2.2.4 Setup Spark	14
2.3 Minimal configuration	15
2.4 Execution	15
2.4.1 Start order (recommended)	15
2.4.2 Make commands	16
2.4.3 Demo mode	16
3 Contracts and interfaces	17
3.1 Kafka topics and routing	17
3.1.1 Topic families	17
3.1.2 Producer / consumer matrix	18
3.1.3 Topic naming rules	20
3.2 Message schemas	20
3.2.1 Serialization adapters (dt/communication/adapters/)	20
3.2.2 Raw sensor reading	21
3.2.3 Processed sensor reading	22
3.2.4 Alert events	22
3.2.5 Queries (REST)	23
3.2.6 Camera snapshots	24
3.2.7 Controller module (control contracts)	24
3.2.7.1 Control mode (ControlMode)	24
3.2.7.2 Action dispatch (ActionDispatch)	25
3.2.7.3 Action command and history (ActionCommand)	25
3.2.7.4 Routines (authoring + compiled representation)	26
3.3 Interfaces (REST + Socket.IO)	27



3.3.1 Database service (REST)	27
3.3.2 Alert engine (REST)	29
3.3.3 Controller service (REST)	30
3.3.4 Webapp (REST)	31
3.3.5 Webapp (Socket.IO)	32
4 Modules	33
4.1 dt/communication	33
4.1.1 Purpose	33
4.1.2 Configuration	33
4.1.3 Components	33
4.2 dt/collector	36
4.2.1 Purpose	36
4.2.2 Inputs and outputs	36
4.2.3 Configuration	36
4.2.4 Components	36
4.2.5 Run command	37
4.2.6 Failure modes and troubleshooting	37
4.3 dt/data/preprocess	39
4.3.1 Purpose	39
4.3.2 Inputs and outputs	39
4.3.3 Configuration	39
4.3.4 Components	39
4.3.5 Spark specificities	40
4.3.6 Run command	41
4.3.7 Failure modes and troubleshooting	41
4.4 dt/alerts (future dt/analytics)	44
4.4.1 Purpose	44
4.4.2 Inputs and outputs	44
4.4.3 Configuration	44
4.4.4 Components	44
4.4.5 REST API surface	45
4.4.6 Run command	45
4.4.7 Failure modes and troubleshooting	45
4.5 dt/data/database	46
4.5.1 Purpose	46
4.5.2 Inputs and outputs	47
4.5.3 Configuration	47
4.5.4 Components	47
4.5.5 REST API surface	47
4.5.6 Run command	47
4.5.7 Failure modes and troubleshooting	48
4.6 dt/webapp	48
4.6.1 Purpose	49
4.6.2 Inputs and outputs	49
4.6.3 Configuration	49
4.6.4 Components	50
4.6.5 Run command	50
4.6.6 Failure modes and troubleshooting	50



4.7	dt/controller	50
4.7.1	Purpose	50
4.7.2	Inputs and outputs	51
4.7.3	Configuration	51
4.7.4	Components	51
4.7.5	Run command	51
4.7.6	Failure modes and troubleshooting	52
5	Configuration	52
5.1	Environment variables (.env)	53
5.2	Configuration responsibilities by module	55
5.3	Preprocessing config (YAML)	55
5.4	Alert rules (YAML)	57
5.5	Actuator policies (YAML)	58
6	Storage	59
6.1	What is stored where	60
6.2	Query patterns	62
6.3	Schema management	62
6.4	Data retention	63
7	Guidelines	64
7.1	General coding standards	64
7.2	Adding a sensor	65
7.2.1	Add the topic (the routing contract)	65
7.2.2	Implement the collector sensor class (the hardware adapter)	66
7.2.3	Wire it into the collector entrypoint (the runnable service)	67
7.2.4	Extend preprocessing configuration (the data quality contract)	67
7.2.5	Extend the dashboard subscriptions and charts (the UX contract)	68
7.3	Adding an alert rule	68
7.3.1	Rule matching: what does source mean?	68
7.3.2	Rule definition structure	68
7.3.3	Persistence and cooldown (how to avoid alert spam)	69
7.3.4	Validation strategy (prove the rule works)	69
7.4	Extending preprocessing	69
7.4.1	Extending an existing stage by adding a strategy	69
7.4.2	Adding a new stage (processor) to the pipeline	70



1 Overview

1.1 Architecture overview

The Digital Twin system is composed of **several services** that work together to collect, process, store, and visualize, and later analyse, sensor data, as well as to trigger alerts based on predefined rules. The data flow begins with the **collector service**, which interfaces with hardware sensors to gather raw readings and publish them to Kafka topics. The **preprocessing service** then consumes these raw readings, applies data quality checks and transformations, and emits processed readings back to Kafka. The **database service** is responsible for persisting these processed readings into a TimescaleDB database, a PostgreSQL extension, and providing a REST API for querying the data. Concurrently, the **alert engine** evaluates the processed readings against user-defined rules, triggering alert events that are also published to Kafka. Finally, the **web application service** consumes both processed readings and alert events to provide real-time updates and historical data visualization through a user-friendly dashboard.

1.1.1 Services and responsibilities

1.1.1.1 Collector service

The `dt/collector` service is responsible for interfacing with the hardware sensors to gather raw readings. It implements a polling loop that periodically collects data from the sensors based on their configured intervals. Additionally, it manages sensor bindings, ensuring that each sensor is correctly identified in the system. Once the data is collected, the service publishes the raw readings to designated Kafka topics for further processing.

1.1.1.2 Preprocessing service

The `dt/data/preprocess` service operates as a Spark Structured Streaming application. It ingests raw sensor readings from Kafka, processes them through a configurable data quality pipeline, and transforms the data into a standardized format. Typical pipeline steps include calibration to correct sensor drift, validation checks to identify outliers or erroneous data, imputation to address missing or invalid values when necessary, and normalization to ensure consistency across different sensor types. All steps are configurable, allowing adaptation to various sensor characteristics, processing strategies, and data quality requirements. Finally, the processed readings are published back to Kafka for consumption by downstream services.

1.1.1.3 Database service

The `dt/data/database` service manages the storage of processed sensor readings and alert events in a TimescaleDB database, which extends PostgreSQL for efficient time-series data handling. Leveraging TimescaleDB, the service supports both time-series and relational data, making it ideal for storing sensor data, alerts, and related metadata. It aggregates data over time to optimize storage and applies data retention policies for long-term management.

The service exposes a REST API for querying historical sensor data, alert and routines information. It also functions as a Kafka bridge, consuming processed readings and alert events from Kafka topics and persisting them in the database. This ensures that all data is stored efficiently and remains accessible for analysis and visualization.

1.1.1.4 Alert engine

The `dt/alerts` service evaluates processed sensor readings against user-defined alert rules, which are typically specified in YAML format. Users can define conditions that trigger alerts based on sensor data. The alert engine continuously consumes processed readings from Kafka,

checks them against these rules, and manages the alert lifecycle including triggering, acknowledging, and resolving alerts.

When an alert condition is detected, the service publishes alert events to a dedicated Kafka topic (`dt.alerts`). It also exposes REST endpoints for managing alert rules, such as submitting alerts from other services, acknowledging active alerts to silence notifications, clearing resolved alerts, and listing all configured rules.

i Info

Next enhancements will be to shift this service to an **Data Analysis service** that integrates AI/ML models for predictive analytics, plant health classification and anomaly detection. This service will then be responsible for both rule-based alerts and AI-driven insights.

1.1.1.5 Web application service

The `dt/webapp` service provides a user-friendly dashboard for visualizing real-time and historical sensor data, as well as managing alerts and setting up routines. It consumes both processed sensor readings and alert events from Kafka topics to provide up-to-date information to users. The web application uses Socket.IO to stream real-time updates to the dashboard, ensuring that users receive the latest data as it becomes available. Additionally, the web application interacts with the database service's REST API to query historical data, allowing users to analyze trends and patterns over time.

🚩 Goal

- In the future, the web application will integrate AI and machine learning models insights for:
 - Predictive analytics such as predict the soil moisture level
 - As well as classification task to determine the health status of a plant based on sensor data
 - ...
- The application will manage system configuration, including:
 - Dynamic Sensor settings (e.g polling intervals)
 - Alerting strategies (e.g., thresholds, notification preferences)
 - AI/ML model parameters and lifecycle states (e.g., training, deployment, versioning)
 - Dynamic Preprocessing pipeline configuration (e.g preprocessing strategies, data quality rules, ...)
 - Simulation mode to test the system with given scenarios

1.1.2 Control service (future)

The `dt/controller` service is responsible for managing actuators and enabling closed-loop control based on sensor data and alerts. It provides an API to control various actuators, such as a water pump, fans, heating or lighting. Users can set up routines that trigger specific actions when certain conditions are met, allowing the system to automatically respond to changes in the environment. For example, if a soil moisture sensor detects low moisture levels, the controller can automatically activate the irrigation system to water the plants. This service enhances the system's ability to maintain optimal conditions without requiring manual intervention.

1.1.3 Main data flows (high level)

Flow of sensor readings through the dt

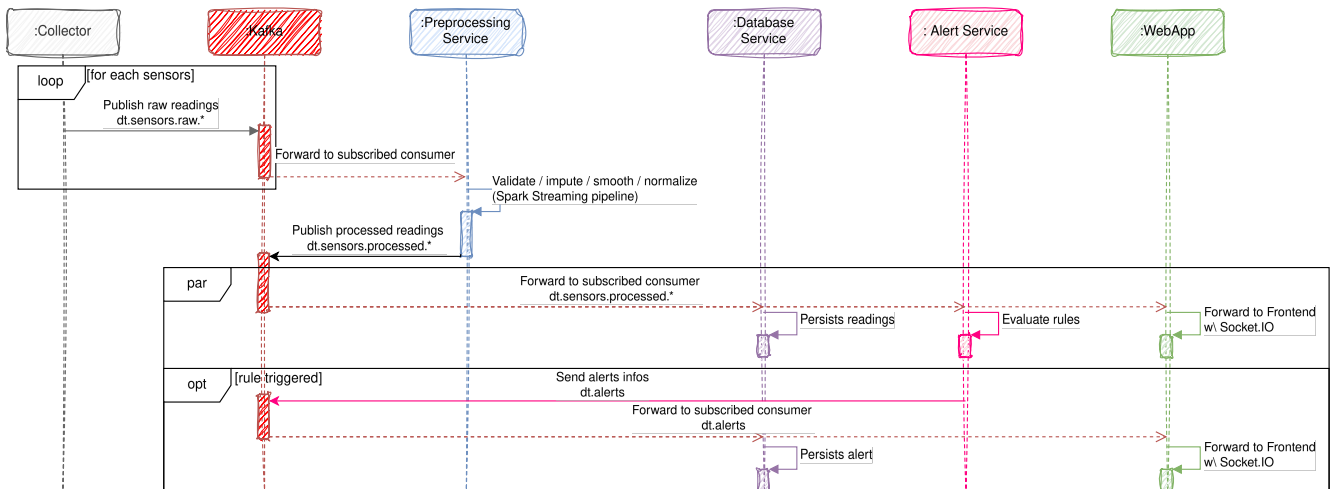


Figure 1 : Sequence diagram illustrating the flow of a sensor reading throughout the whole dt system.

Flow of user interaction through the dt

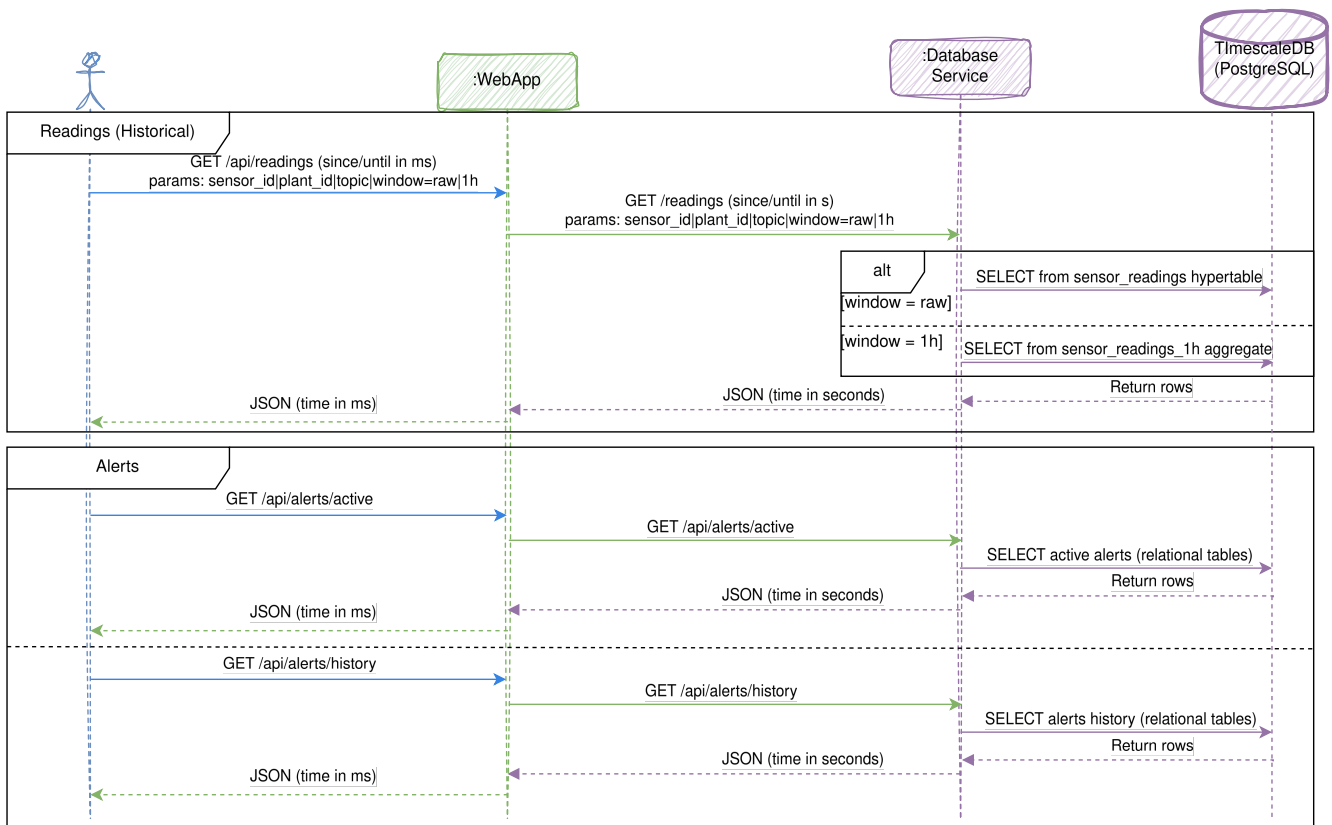


Figure 2 : Sequence diagram illustrating the data flow when the user interact with the dashboard to 1) get the history, over a specified period of time, of the sensor readings and 2) get the either the active alerts or the history of alerts.

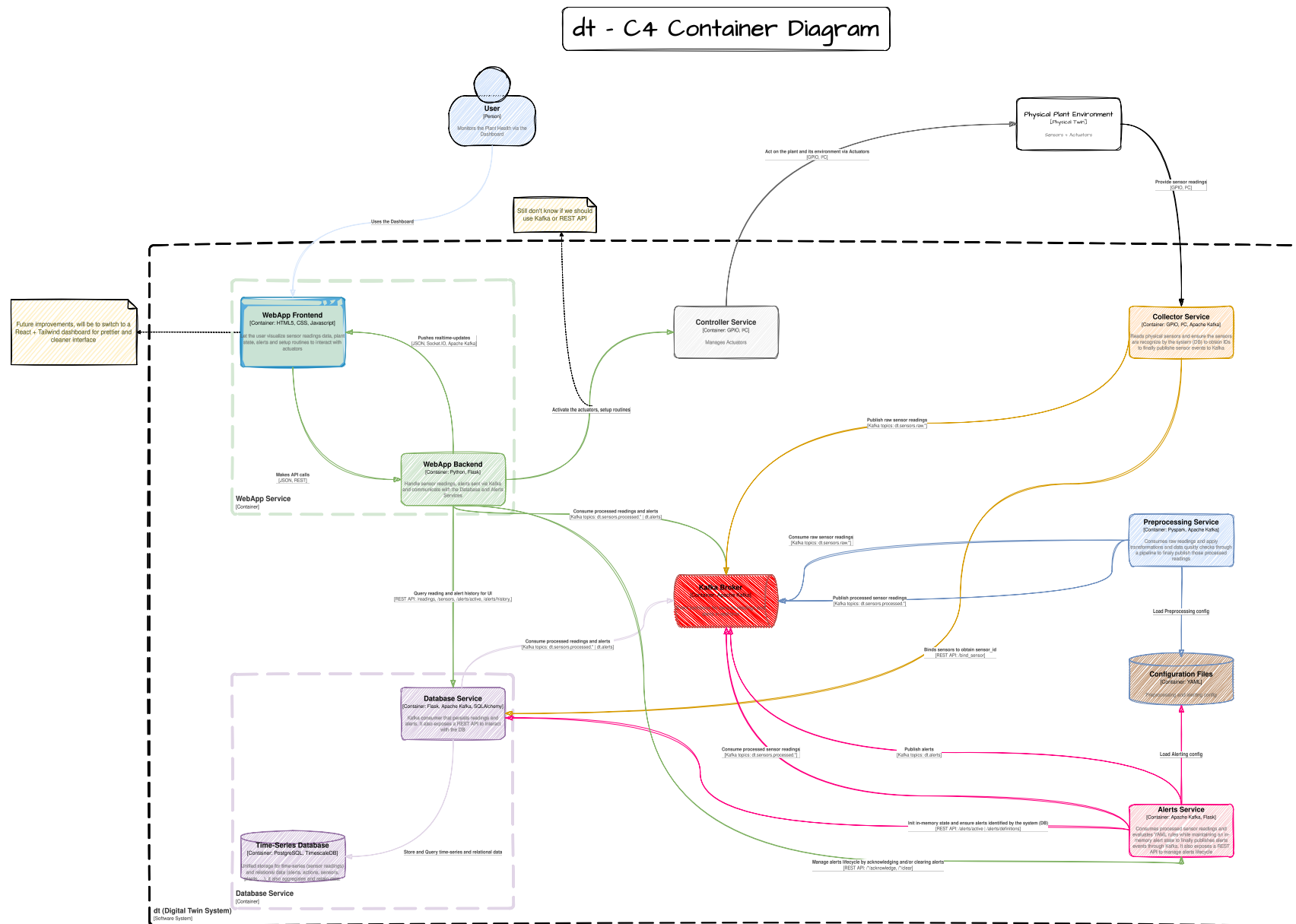


Figure 3 : High-level architecture diagram illustrating the main services and data flows within the Digital Twin system.

ULB - Digital Twin - Runbook

1.2 Conventions (time, units, naming)

This section outlines the conventions used across the project for time representation, units of measurement, and naming of topics and fields. Adhering to these conventions ensures consistency and clarity in communication between different components of the system.

1.2.1 Time representation

Time is represented in different formats across the system. The backend dataclasses and database/API use Unix epoch seconds (as a float) in the dataclass fields. In contrast, the web UI uses milliseconds. It is important to explicitly state where conversions between these formats occur and what fields are expected on each interface to avoid confusion and ensure proper handling of time data. The conversion for the web UI is typically done by multiplying the Unix epoch seconds by 1000 to convert it to milliseconds (`dt/webapp/api.py:20:convert_timestamp_for_browser`), and the conversion for the backend is done by dividing the milliseconds by 1000 to convert it back to seconds (`dt/webapp/api.py:73`).

i Info

I might consider standardizing on one time format across the system in the future to simplify handling and reduce the chances of errors due to incorrect conversions. However, for now, it is important to be aware of these conventions and ensure that all components adhere to them when processing time data.

1.2.2 Units

Units of measurement for sensor data are standardized across the system to ensure consistency and ease of interpretation. The following units are used for the respective sensor types:

- Temperature: degrees Celsius (°C)
- Humidity: percentage (%)
- Light: lux
- Soil moisture: percentage (%) after calibration. Note that the sensor returns values in a different format before calibration. The sensor returns a value from 0 to 2000, which is then calibrated to a percentage value for easier interpretation and use in the system.

2 Quickstart

In this section, we provide a quickstart guide to get the Digital Twin system up and running. This guide covers the prerequisites, installation guidelines, minimal configuration, recommended start order of services, and smoke checks to verify that the system is functioning correctly.

2.1 Prerequisites

The reference deployment for this project is a Raspberry Pi 4B running a Debian-based OS (Raspberry Pi OS). However, the software stack is not tied to a specific board: if you can run Python ≥ 3.11 and reach Kafka + PostgreSQL, you can run most modules on any Linux machine.

2.1.1 Hardware

Hardware is required when you want to collect real readings from sensors and drive actuators. If you are working on the dashboard or documentation, you can use the dashboard demo mode and skip the physical setup entirely.

The current prototype uses the following components:

- **Core platform:** Raspberry Pi 4B (or equivalent Linux host), breadboard, jumper wires, and a stable power supply.
- **Sensors:** DHT22 (temperature/humidity), BH1750 (ambient light), and an Adafruit STEMMA soil moisture sensor.
- **Actuators:** a 4-channel relay module, a small water pump, a 12V LED strip, a PTC heater, and a 12V fan.
- **Camera :** Raspberry Pi Camera Module for snapshots.

It is important to ensure that all hardware components are properly wired to the expected pins on the Raspberry Pi. The following figure provides a wiring overview, and the next subsection lists every connection explicitly so you can double-check your assembly.

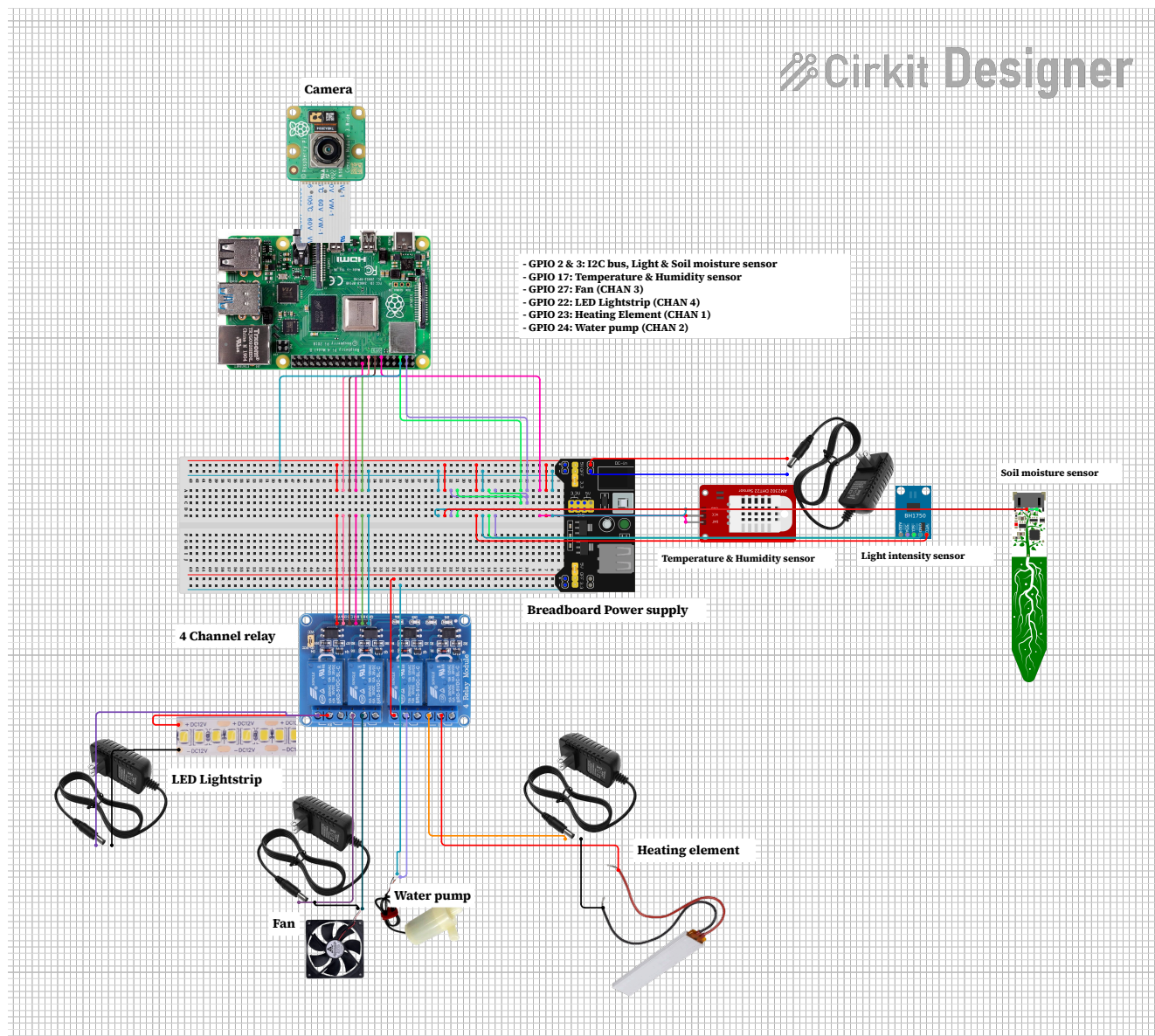


Figure 4 : Wiring diagram illustrating the connections between the Raspberry Pi and the sensors and actuators in the Digital Twin system. The diagram shows how each sensor and actuator is connected to the appropriate GPIO pins on the Raspberry Pi, as well as the power supply connections for each component.

To complement the above diagram, the following section provides a component-by-component breakdown of all wiring connections. This detailed list is intended to help you verify each

connection during assembly and ensure that every sensor and actuator is correctly integrated into the system.

2.1.1.1 Wiring Details

This wiring guide follows two simple principles. First, all I²C sensors share the same bus: both the BH1750 and the STEMMA soil sensor are connected to the standard Raspberry Pi I²C pins (SDA on **GPIO 2**, SCL on **GPIO 3**). Second, power is kept intentionally simple: a single **5V rail** is used for logic (sensors and the relay module input side), while each 12V actuator is powered by its own 12V power supply.

On the low-voltage side, the Raspberry Pi, sensors, and the relay module input must share a common ground so the GPIO signals are referenced correctly. The 12V actuator circuits are switched through the relay contacts and can remain electrically isolated from the Raspberry Pi ground, since the relay provides a mechanical separation between the control side and the actuator device side.

The relay module is used as an electrically-isolated switch: its input pins (IN1-IN4) are driven by Raspberry Pi GPIOs, and its output contacts route power to the actuator devices. Whether you connect the supply to **COM** and the actuator device to **NO** (or the reverse) does not change functionality, as long as you keep the circuit consistent for each channel.

Raspberry Pi 4B

- **GPIO 2 (SDA)**: Connected to Adafruit STEMMA Soil Sensor SDA and BH1750 SDA
- **GPIO 3 (SCL)**: Connected to Adafruit STEMMA Soil Sensor SCL and BH1750 SCL
- **GPIO 17**: Connected to the DTH22 sensor data pin
- **GPIO 27**: Connected to Relay 4 Channel IN2 (actuator channel 3)
- **GPIO 22**: Connected to Relay 4 Channel IN3 (actuator channel 4)
- **GPIO 23**: Connected to Relay 4 Channel IN3 (actuator channel 1)
- **GPIO 24**: Connected to Relay 4 Channel IN4 (actuator channel 2)
- **GND**: Connected to MB102 Breadboard Power Supply Module GND

MB102 Breadboard Power Supply Module 3.3V/5V

- **VCC**: Connected to a stable **+5V** source
- **GND**: Connected to the common ground (shared with the Raspberry Pi)
- **3.3V**: Not used in this configuration
- **5V**: Powers the relay module logic and the 5V devices (e.g., pump)

12V Power Supplies (one per actuator)

- **+12V**: Routed through relay contacts to the corresponding actuator device
- **GND**: Connected directly from the power supply to the actuator's GND terminal.

Relay 4 Channel 5V

- **GND**: Connected to MB102 GND (common ground)
- **VCC**: Connected to MB102 5V rail
- **IN1**: Connected to Raspberry Pi GPIO 23
- **IN2**: Connected to Raspberry Pi GPIO 24

- **IN3:** Connected to Raspberry Pi GPIO 27
- **IN4:** Connected to Raspberry Pi GPIO 22
- **Channel 1 (heater, 12V):** routes +12V from its supply to the heater positive lead
- **Channel 2 (pump, 5V):** routes the MB102 5V rail to the pump positive lead
- **Channel 3 (fan, 12V):** routes +12V from its supply to the fan positive lead
- **Channel 4 (LED strip, 12V):** routes +12V from its supply to the LED strip positive lead

DHT22 sensor

- **GND:** Connected to MB102 Breadboard Power Supply Module GND
- **VCC:** Connected to MB102 Breadboard Power Supply Module VCC
- **DAT:** Connected to Raspberry Pi GPIO 17

BH1750

- **VCC:** Connected to MB102 Breadboard Power Supply Module VCC
- **GND:** Connected to MB102 Breadboard Power Supply Module GND
- **SDA:** Connected to Raspberry Pi GPIO 2 SDA
- **SCL:** Connected to Raspberry Pi GPIO 3 SCL

Adafruit STEMMA Soil Sensor

- **GND:** Connected to MB102 Breadboard Power Supply Module GND
- **VIN:** Connected to MB102 Breadboard Power Supply Module VCC
- **SDA:** Connected to Raspberry Pi GPIO 2 SDA
- **SCL:** Connected to Raspberry Pi GPIO 3 SCL

PTC heater

- **Live:** Connected to Relay channel 1 output contact
- **Neutral:** Connected to the heater 12V power supply GND

5V Mini Water Pump

- **RED:** Connected to Relay channel 2 output contact
- **BLACK:** Connected to MB102 GND

12V fan

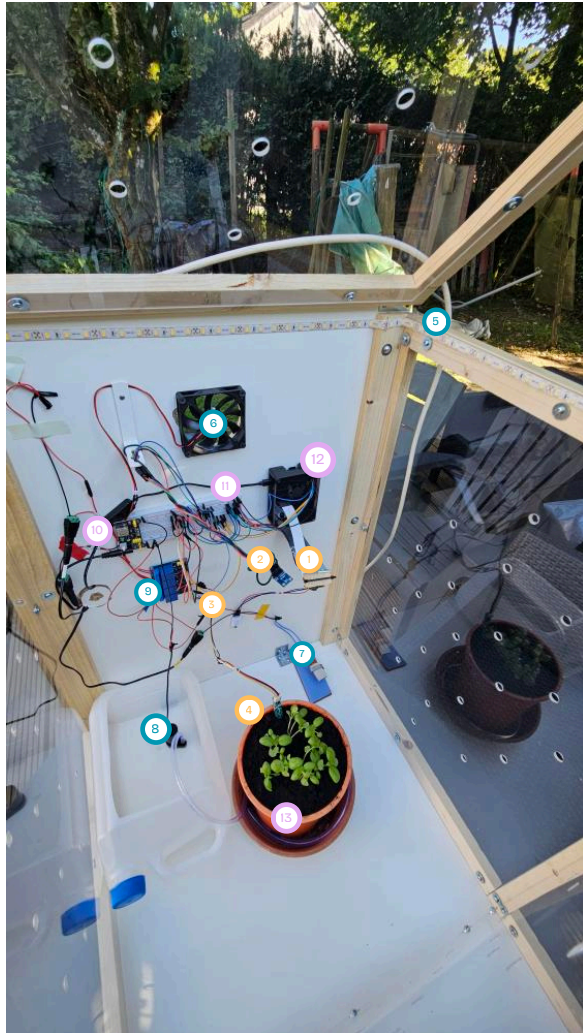
- **RED:** Connected to Relay channel 3 output contact
- **BLACK:** Connected to the fan 12V power supply GND

12V White LED Strip

- **+12V:** Connected to Relay channel 4 output contact
- **GND:** Connected to the LED strip 12V power supply GND



Figure 5 : Photo of the assembled Digital Twin system, showing the Raspberry Pi connected to the sensors and actuators through the breadboard and relay module.



- | | | |
|---|---|------------------|
| <ul style="list-style-type: none"> 1 Camera 2 Light intensity sensor 3 Temperature & Humidity sensor 4 Soil moisture sensor | } | Sensors |
| <ul style="list-style-type: none"> 5 LED Lightstrip 6 Fan 7 Heating element 8 Water pump 9 4 Channel relay | } | Actuators |
| <ul style="list-style-type: none"> 10 Breadboard power supply 11 Breadboard 12 Raspberry Pi 13 Basil (Physical Twin) | } | Structure |

Figure 6 : Photo of the assembled Digital Twin system with the box open, showing the internal wiring and connections between the Raspberry Pi, sensors, and actuators with annotations highlighting the key components.

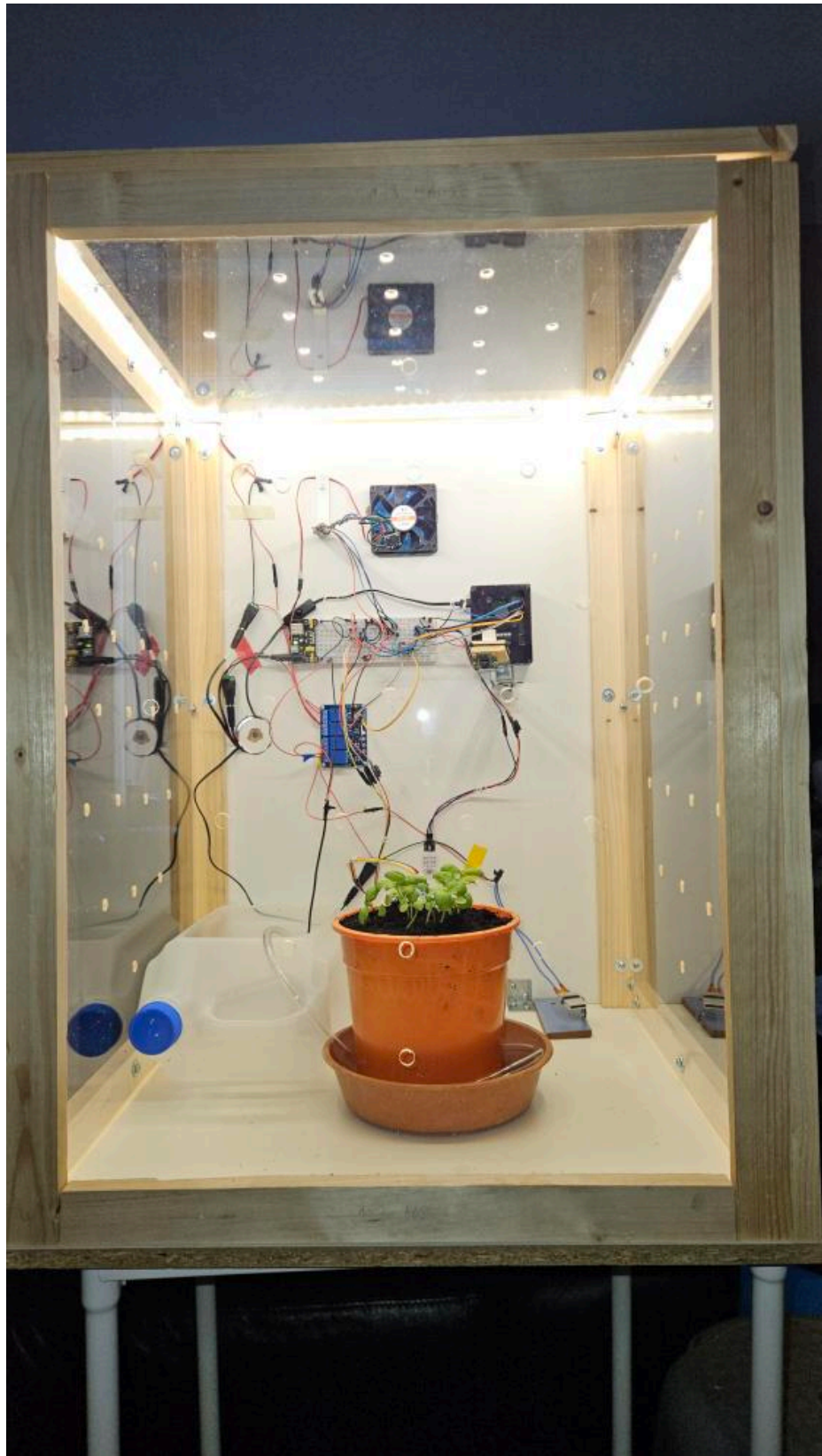


Figure 7 : Photo of the assembled Digital Twin system at night, demonstrating the operation of the 12V White LED Strip providing additional lighting for the plants.

2.1.2 Software

The stack consists of Python services (collector, controller, database service, alert engine, and dashboard) plus two external dependencies: Kafka for messaging and PostgreSQL/TimescaleDB

for persistence. The easiest way to work with the repository is to install dependencies via Poetry and to run each service via `poetry run` (or the provided Make targets).

At minimum, you need:

- Python 3.11+ and Poetry to install and run the services.
- A Kafka broker reachable at `KAFKA_URL` (local or remote).
- A PostgreSQL instance with the TimescaleDB extension reachable at `PG_DATABASE_URL`.

Spark is only required for the preprocessing module. If you run preprocessing, or have the Kafka broker running on your machine, you also need a Java runtime (Java 17 is a good default on Debian-based systems), and you must ensure that Spark has access to the Kafka connector packages at runtime.

2.2 Installation guidelines

This section describes a pragmatic installation path for the stack. The key idea is to separate (1) the Python environment managed by Poetry from (2) the infrastructure services the system depends on (Kafka, PostgreSQL/TimescaleDB and Spark), so you can install and manage them independently. This also allows you to run the Python services on a different machine than the infrastructure if needed.

2.2.1 Poetry profiles

This repository is a single Poetry project with optional dependency groups that let you tailor the installation to your environment. The goal is to keep the default install lightweight, while still providing a setup for development.

At a high level, you will typically choose between:

- a developer workstation install (tests + database + Spark),
- a Raspberry Pi runtime install (GPIO/I²C sensor libraries + database),
- or a minimal install when you only want to work on a specific module.

The available Poetry groups are defined in `pyproject.toml`:

- `dev` for tooling (pytest, ruff, mypy, etc.),
- `db` for PostgreSQL/TimescaleDB access (`sqlalchemy`, `psycopg`),
- `spark` for the preprocessing job (`pyspark`, `pyarrow`),
- `rpi` for Raspberry Pi hardware drivers (CircuitPython libraries, GPIO).

For convenience, the repository exposes Make targets that map to common Poetry installs (see `Makefile`). On a fresh machine, the typical workflow is to pick the right profile, install dependencies, and then run all commands through `poetry run` so the correct virtual environment is used.

Recommended profiles:

- `make install-dev` for development on a non-Raspberry Pi machine.
- `make install-rpi` on the Raspberry Pi (installs the GPIO/I²C dependencies).
- `make install-db` if you only want the database service and its dependencies.
- `make install-spark` if you only want the Spark preprocessing job.

If you already have Poetry installed but the virtual environment targets the wrong Python interpreter, you can force it to use a specific version (the project requires Python ≥ 3.11): `poetry env use python3.11`.

2.2.2 Setup Kafka (KRaft)

Kafka is used as the messaging backbone of the system: the collector publishes raw sensor events, the preprocessing job consumes them and republishes processed events, and other

modules (database bridge, alert engine, dashboard) subscribe to the relevant topics. For this prototype, a single-node Kafka broker running in KRaft mode (i.e., without ZooKeeper) is sufficient.

i Info

Kafka historically relied on ZooKeeper for cluster metadata management, but KRaft (Kafka Raft) is the built-in successor that removes the need for an external ZooKeeper cluster. KRaft is a good fit here because it keeps the deployment lightweight and easier to operate, which aligns with a resource-constrained Raspberry Pi setup, while still providing the features we need (topics, partitions, consumer groups).

If ingestion volume grows (more sensors, higher sampling rates) or higher availability becomes a requirement, the Kafka deployment should be upgraded to multiple nodes in KRaft mode: a controller quorum (typically 3 nodes) and multiple brokers with replication enabled (e.g., replication factor 3). In addition, partition counts and producer/consumer settings should be tuned to increase parallelism and maintain throughput under load.

On Debian-based systems (including Raspberry Pi OS), the repository provides a provisioning script at `scripts/setup_kafka.sh`. The script installs Java, downloads Kafka under `/opt/kafka`, formats the KRaft log directory, and registers a `kafka-kraft` systemd service.

Two practical notes before running it:

- The script attempts to discover both a local IP address and a public IP address (via `ifconfig.me`) to configure `advertised.listeners`. If the host does not have internet access, or if you are on a private LAN with no public address, you may need to adapt the script or edit `/opt/kafka/config/kraft/server.properties` after installation.
- The project services usually connect to Kafka through `KAFKA_URL` (default `localhost:9092`). As long as the broker is reachable at that address from the services, the rest of the stack remains unchanged.

After installation, verify the broker is reachable before moving on:

- Check service status: `sudo systemctl status kafka-kraft.service`
- Follow logs: `sudo journalctl -u kafka-kraft.service -f`
- Confirm the listener is up: `nc -z localhost 9092`
- List topics (should succeed even if empty):
`/opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list`

Kafka is configured with `auto.create.topics.enable=true`, which is convenient for local development. In a more controlled setup, you can explicitly create the project topics (raw and processed sensor topics, plus `dt.alerts`) using the Kafka CLI tools shipped in `/opt/kafka/bin/`.

In addition to the upstream Kafka shell tools, the repository provides a small helper CLI in `scripts/kafka_manager.py`. It wraps `kafka-topics.sh` and `kafka-configs.sh` and is useful when you want a single entry point to list, describe, create, delete, or alter topics without remembering the full command syntax.

If you use it, treat the `list`, `describe`, `create`, `delete`, and `config` commands as the primary interface. The `setup` command exists, and it currently creates the reference topics needed by the stack: `dt.sensors.raw.*`, `dt.sensors.processed.*`, and `dt.alerts` etc. If you later introduce an other topics, you can extend the script (and `dt/communication/topics.py`) to include it.

2.2.3 Setup PostgreSQL/TimescaleDB

The Digital Twin persists both time-series readings and relational domain data into PostgreSQL with the TimescaleDB extension. The recommended approach for a local Raspberry Pi setup is to install PostgreSQL and TimescaleDB directly on the host and to keep the database on the same machine as the services.

For Debian-based systems, the repository includes `scripts/setup_postgresql.sh`, which automates the installation and performs an initial bootstrap:

- installs PostgreSQL and TimescaleDB packages,
- creates a database and an application user,
- enables the `timescaledb` extension in that database,
- writes `PG_DATABASE_URL` into a local `.env` file.

The script is idempotent and accepts environment variable overrides, so you can adjust the database name/user/password without editing the file (for example by setting `DB_NAME`, `DB_USER`, `DB_PASS`, `DB_HOST`, `DB_PORT`).

Once PostgreSQL is installed, verify connectivity and the TimescaleDB extension before proceeding to migrations:

- ensure the service is running: `sudo systemctl status postgresql`
- connect to the database with `psql` using the credentials you configured
- confirm TimescaleDB is enabled by running `CREATE EXTENSION IF NOT EXISTS timescaledb;` (the setup script already does this) and checking that the extension exists.

In the application code, `PG_DATABASE_URL` is expected in SQLAlchemy form (default `postgresql+psycopg://...`).

2.2.3.1 Database Migrations

The database schema is managed through SQL migrations stored under `dt/data/database/migrations/`. Migrations are applied in alphanumeric order and are tracked in a dedicated `schema_migrations` table, which makes the process safe to re-run and suitable for automated startup checks.

There are two supported ways to run migrations:

First, the database service runs migrations on startup (see `dt/data/database/app.py`). This is the most convenient option when you start the stack via `make run-database` and want the schema to always be present.

Second, you can run migrations manually using the CLI runner: `poetry run python scripts/run_sql_migration.py`. This is useful during installation because it lets you validate database connectivity and permissions in isolation, before you bring up the whole stack.

If you need to point the runner at a non-default migrations directory, set `DB_MIGRATIONS_DIR` (defaults to `dt/data/database/migrations`). If migrations fail, the most common causes are a wrong `PG_DATABASE_URL`, missing privileges for the configured user, or a missing TimescaleDB extension.

2.2.4 Setup Spark

Spark is only required for the preprocessing module. If you are not running the preprocessing job, you can skip this section and avoid installing the `spark` Poetry group.

From the Python side, Spark support is provided by `pyspark`, installed via the `spark` Poetry group (for example `make install-spark` or `make install-dev`). From the system side, you still need a working Java runtime (Java 17 is a good default on Debian-based systems).

The preprocessing job uses Spark Structured Streaming with Kafka sources and sinks (`spark.readStream.format("kafka")` and `.writeStream.format("kafka")`). This requires the Spark Kafka connector JARs to be available at runtime. A simple way to do this with PySpark is to set `PYSPARK_SUBMIT_ARGS` to include the `spark-sql-kafka-0-10` package matching your Spark/PySpark version.

For example, you can retrieve the PySpark version with :

```
python -c "import pyspark; print(pyspark.__version__)"
# And then export the submit args with the correct version:
PYSPARK_SUBMIT_ARGS="--packages org.apache.spark:spark-sql-kafka-0-10_2.12:${PYSPARK_VERSION}"
pyspark-shell"
```

This makes the Kafka source/sink available to the job without requiring a separate Spark installation, as long as Java is present.

Finally, make sure the checkpoint directory is configured and writable. By default, the project uses `PREPROCESSING_CHECKPOINT_DIR=./spark-checkpoints/preprocessing`. Checkpoints are required for streaming correctness (offset tracking and state recovery), so avoid placing them on ephemeral storage.

As a basic smoke test that Spark itself runs, the repository includes a small hands-on script under `scripts/hands-on-scripts/software/spark/` (it does not exercise Kafka, but it validates that your PySpark + Java setup is functional).

2.3 Minimal configuration

To run the Digital Twin system, you need to configure the following minimal set of environment variables in a `.env` file at the root of the project:

- `KAFKA_URL` : The URL of the Kafka broker that the system will connect to
- `PG_DATABASE_URL` : The connection string for the PostgreSQL/TimescaleDB database that the system will use for storing and querying data
- `FLASK_DASHBOARD_URL` : The URL of the Flask dashboard that the web application will connect to for fetching data and alerts
- `FLASK_DB_URL` : The URL of the Flask database that the web application will connect to for fetching data and alerts
- `PREPROCESSING_CONFIG_PATH` : The file path to the configuration file for the preprocessing job, which contains settings for each steps of the preprocessing pipeline.
- `FLASK_CONTROLLER_URL` : The URL of the Flask controller that the control module will connect to for sending actuator commands.

2.4 Execution

2.4.1 Start order (recommended)

The recommended start order is mostly about dependencies: bring up infrastructure first, initialise the schema early, and then start consumers before producers so you do not lose the first events published on the broker.

Start by ensuring Kafka is running and reachable at `KAFKA_URL` (default `localhost:9092`). Kafka is the event backbone: it carries sensor readings to downstream consumers and transports alert lifecycle events to the dashboard and the database bridge.

Next, start PostgreSQL/TimescaleDB and then the database service. The database service is both an HTTP API and a Kafka bridge: on startup it applies SQL migrations (so the schema exists), then it consumes incoming events and writes them to the database. If you start other modules before the database service is ready, they may still boot but will fail once they try to persist or query state.

The preprocessing job needs to run after the database service as it will start by querying the database for sensor metadata to setup the processing pipeline. It also needs to be run before the other consumers as it takes some times to start up and subscribe to the raw sensor topics, and you want it to be ready to process events as soon as the collector starts publishing.

If you want alerting, start the alert engine after the database service. It subscribes to processed sensor topics, evaluates rules, and publishes alert events to `dt.alerts`. The dashboard can then display alerts in real time, while the database service persists them for history and auditing.

The controller module also depends on the database service, as it needs to bind the actuators to the database state and to fetch the active routines before making control decisions. Starting it after the database service ensures that it can connect successfully and that the API endpoints it relies on are available.

Finally, start the web dashboard and then the collector. Starting the dashboard first is useful because it immediately shows whether it can connect to Kafka and seed alert state via the database API. Starting the collector last ensures that as soon as it begins publishing, the consumers (database bridge, dashboard, controller and alert engine) are already subscribed and ready.

2.4.2 Make commands

The repository is meant to be executed from the project root with Poetry, and the Makefile provides thin convenience targets that run the correct Python entry points through `poetry run`. This keeps execution consistent across machines and avoids accidentally using the system Python interpreter.

For day-to-day use, it is easiest to start each long-running service in its own terminal (or via a terminal multiplexer). A typical local run starts the database service with `make run-database`, then starts alerting with `make run-alert-engine`, and then starts the UI with `make run-dashboard`. Once consumers are running, you can start producing data with `make run-collector`.

If you only want to run a subset of the stack, you can treat each module as an independent process. For example, the dashboard can run without Kafka (it will show a disconnected status), and the alert engine can be started later as soon as processed data is flowing.

i Info

A future script will be added to orchestrate the startup of all services in the correct order, keeping them running with logs.

2.4.3 Demo mode

The dashboard includes a demo mode that emits synthetic processed readings and alerts directly over Socket.IO. This mode is useful when you want to work on the frontend, record screenshots, or demo the UI without having Kafka and the hardware available.

Demo mode is enabled via CLI flags on the dashboard entry point. When `--demo` is enabled, the dashboard does not start its Kafka consumer and instead generates periodic updates at the configured interval. A typical command is: `poetry run python -m dt.webapp.app --demo --demo-interval 1.0`.

Demo mode only replaces the real-time stream. The dashboard still exposes API endpoints that proxy to the database service for historical queries and alert history. If the database service is not running, the UI will still load and the live demo stream will work, but history-related features may fail until you start `make run-database`.

🚩 Goal

We could also consider adding a demo mode to each services (independent of the dashboard) that mocks its inputs and outputs. For example, a collector demo mode could publish synthetic readings to Kafka, while a database service demo mode could serve historical queries from a static JSON file. This would allow you to test each module in isolation without needing the full stack running.

3 Contracts and interfaces

This section is important for interoperability between modules. It defines the reference Kafka topic families and routing rules, the on-wire message schemas for the main dataclasses, and the REST/Socket.IO interfaces exposed by the services.

If you are adding a new service, extending an existing one, or debugging a cross-service issue, you should be able to start here and answer three questions quickly: **(1)** which topic(s) carry the data you need, **(2)** what payload shape and field semantics to expect, and **(3)** which HTTP/WebSocket entry points are available for queries and control.

3.1 Kafka topics and routing

3.1.1 Topic families

Kafka is the backbone of the system: it decouples producers and consumers, enables fan-out to multiple downstream modules, and provides buffering when one component is temporarily offline. Because the system is distributed, topic names are treated as a **contract**: once a topic is published, downstream modules can only remain compatible if the topic, the payload schema, and the routing rules stay stable. In other words, if a topic name changes or starts carrying different payloads, all producers and consumers that use that topic must be updated together to avoid runtime errors.

The reference topic names are defined in `dt/communication/topics.py` and are built around one core idea: each sensor measurement has a **semantic topic** (the sensor type or what the **data represents**), and each processing stage has its own **transport topic**.

At the semantic layer, sensor types live under the base namespace: `dt.sensors.<type>`. The `<type>` segment is a stable, domain-level identifier such as `temperature` or `soil_moisture`.

At the transport layer, stage-specific variants are derived by inserting a single segment:

- `dt.sensors.raw.<type>` carries raw, unprocessed readings.
- `dt.sensors.processed.<type>` carries validated / calibrated / normalized readings and their quality metadata.

Alerts are emitted on a dedicated, non-sensor topic: `dt.alerts`. This topic carries alert lifecycle events (created/updated/acknowledged/cleared) and is consumed by modules such as the dashboard and the database.

i Info

I chose to keep raw and processed streams separate at the transport level to give downstream modules the flexibility to choose which one they want without having to filter or transform the stream. This also keeps the routing simpler and more predictable, since the presence of `raw` or `processed` in the topic name is a clear signal of the data quality and processing stage. The payload `topic` field still indicates the sensor type, so the semantic meaning of the reading is preserved across stages.

3.1.2 Producer / consumer matrix

The following matrix describes the responsibilities of each module. In Kafka terms, each service runs its own consumer group, so multiple services can consume the same topic independently (fan-out) without “stealing” messages from each other.

Table 1 : Kafka topic responsibilities by module.

Module	Consumes (Kafka)	Publishes (Kafka)	Notes
Collector	(none)	dt.sensors.raw.<type>	Publishes RawSensorData payloads. The payload topic is dt.sensors.<type>.
Controller	dt.sensors.processed.<type>	dt.actions	Consumes processed readings for routine control logic and publishes action commands.
Preprocess (Spark)	dt.sensors.raw.*	dt.sensors.processed.<type>	Consumes raw readings, applies validation/imputation/smoothing/calibration/normalization, then emits ProcessedSensorData.
Alert engine	dt.sensors.processed.<type>	dt.alerts	Evaluates alert rules on processed readings and publishes alert lifecycle events.
Database	dt.sensors.processed.<type>, dt.alerts, dt.actions	(none)	Persists processed readings to TimescaleDB and alert events to relational audit/history tables.
Webapp	dt.sensors.processed.<type>, dt.alerts, dt.actions	(none)	Relays processed readings and alert events to Socket.IO (browser timestamps in ms).
Analytics / AI (future)	dt.sensors.processed.<type>, dt.alerts, dt.actions	(optional)	Consumes cleaned streams for modelling, feature extraction, and derived indicators.

3.1.3 Topic naming rules

Topic names are part of the system's interface. The naming rules below are designed to keep routing predictable (so simple substring checks and wildcard subscriptions work) and to keep schemas cohesive (so a topic does not accidentally become a “grab bag” of unrelated payloads).

All topic names follow these constraints:

- **Namespace and separators.** Topics use dot-separated namespaces and always start with `dt`. Sensor measurements live under `dt.sensors`. You can consider it like a URL or a filesystem path: the segments are meaningful and should be kept stable. For example, `dt.sensors.raw.temperature` is a valid topic, while `dt.sensors.temperature_raw` is not, because it breaks the convention of having the stage as a separate segment.
- **Stage segments are reserved words.** The tokens `raw` and `processed` are reserved for the stage namespace and must appear exactly as `dt.sensors.raw.<type>` and `dt.sensors.processed.<type>`. Downstream services rely on these exact segments for routing.
- **Sensor type identifiers are stable.** The `<type>` segment is lowercase snake_case (for example `soil_moisture`, `light_intensity`). Introducing a new sensor type means adding it to the reference list in `dt/communication/topics.py` rather than inventing ad-hoc topic strings in individual modules.
- **One schema family per topic.** A topic should carry exactly one payload family (raw sensor, processed sensor, or alert event). If the meaning or required fields of a payload must change in an incompatible way, define a new topic or version the payload explicitly rather than silently mutating the existing contract.

When in doubt, treat `dt/communication/topics.py` as the single source of truth and update the documentation together with any topic changes (a typing mistake or slight inconsistency can always happen). This keeps producers, consumers, and operational scripts aligned.

For local deployments, `scripts/kafka_manager.py setup` can be used to create the full set of reference topics in the broker. The command derives its topic list from `Topics`, which is precisely why keeping `Topics` aligned with this reference matters: changing topic names in one place should be enough to keep the runtime and the operational tooling in sync.

3.2 Message schemas

This section defines the message schemas that cross process boundaries. The same dataclasses are reused in three places: **(1)** Kafka messages (JSON payloads), **(2)** REST APIs (JSON bodies and query parameters), and **(3)** persistence layer (rows in TimescaleDB and relational tables). The goal is to keep these contracts stable so that modules can evolve independently as long as they continue to speak the same “language”.

3.2.1 Serialization adapters (`dt/communication/adapters/`)

The dataclasses in `dt/communication/dataclasses/` describe the **domain objects** the system exchanges. In practice, those objects must be represented in several concrete formats: JSON dictionaries for Kafka and REST, database row payloads for TimescaleDB inserts and selects, Spark rows for DataFrame operations, and compact tuples for stateful streaming.

The `dt/communication/adapters/` package provides a single, explicit place for those transformations. Instead of letting each module reinvent its own “to dict”, “from row”, and “Spark-safe” conversions, the code uses two small entry points: `dump("<format>", obj)` to serialize, and `load("<format>", Class, data)` to deserialize (`dt/communication/adapters/registry.py`).

This layer is not an abstraction for its own sake, it solves several practical problems:

- It keeps payload shapes consistent across modules. If the Kafka consumer in the webapp and the persistence layer in the database service both receive `ProcessedSensorData`, they both rely on the same serializer and the same type coercion rules.
- It prevents leaking storage and runtime details into domain objects. For example, Spark cannot use enums as map keys, and the database schema prefers compact strings for topics and flags. The adapters handle those constraints without changing the core dataclasses.
- It reduces duplication and drift. When a field is added to a dataclass, the conversion logic is updated once in the adapter rather than in every producer/consumer.

Concretely, the project uses four formats:

- `generic`: JSON-safe Python types via `cattrs` used at Kafka and REST boundaries (for example in `dt/communication/messaging_service.py` and `dt/communication/db_client.py`).
- `db_row`: database-oriented payloads, used by the storage layer (`dt/data/database/timescale_storage.py`) and supports both simple rows (readings, sensors) and multi-table alert inserts.
- `spark_row`: conversions to/from `pyspark.sql.Row` for DataFrame processing; used in the pre-processing code that bridge between Spark and dataclasses.
- `tuple`: compact tuple payloads for Spark state storage; used by the preprocessing state provider to keep `GroupState` serialization simple.

As an example, Kafka publishing does not need to know the details of enums or nested dataclasses, it publishes the canonical “generic” representation:

```
from dt.communication.adapters import dump, load

payload = dump("generic", processed_reading) # dict suitable for JSON
reading = load("generic", ProcessedSensorData, payload) # type-coerced dataclass
```

In other words, adapters let the rest of the code talk in terms of domain objects, while still meeting the constraints of each transport or runtime.

3.2.2 Raw sensor reading

Raw readings represent the sensor signal as close as possible to what the hardware provides, with minimal interpretation. On Kafka, these readings are transported on `dt.sensors.raw.<type>` topics, while the `topic` field inside the payload remains the semantic base topic `dt.sensors.<type>` (see “Kafka topics and routing”).

The reference schema is the `RawSensorData` dataclass (`dt/communication/dataclasses/raw_sensor_data.py`). It is intentionally small because it sits at the very beginning of the pipeline: every downstream consumer inherits its meaning.

The required fields and their semantics are:

- `plant_id`: relational identifier for the plant being monitored.
- `sensor_id`: relational identifier for the sensor that produced the reading.
- `timestamp`: Unix epoch timestamp in **seconds** (float), representing when the reading was taken.
- `value`: numeric reading in the unit specified by `unit`.
- `unit`: unit string as displayed to operators (for example `°C`, `%`, `lux`).
- `topic`: semantic base topic (`dt.sensors.<type>`) identifying the measurement type.
- `correlation_id`: opaque trace identifier for end-to-end observability.

The `correlation_id` is generated once at the collector (as a UUID string in the collector via `dt/utls/ids.py`) and then propagated unchanged by preprocessing, alerting, and storage. Con-

sumers should treat the field as a required part of the contract and include it in logs, persisted rows, and emitted alert events.

3.2.3 Processed sensor reading

Processed readings are the output of the preprocessing pipeline and are the default input for most downstream modules (dashboard, alerting, persistence, analytics and control). They are transported on `dt.sensors.processed.<type>` topics as JSON payloads.

The canonical schema is `ProcessedSensorData` (`dt/communication/dataclasses/processed_sensor_data.py`). It extends `RawSensorData` with the minimal set of fields needed to make data quality explicit, auditable, and actionable.

The additional fields are:

- `flags`: a map of validation flags to booleans. A value of `true` indicates a violation for that flag (for example `range_violation=true`), and `valid_data_point` is set to `false` when one or more violations are present.
- `dq_score`: a weighted data-quality score on the interval `[0, 1]`, where `1` indicates a fully passing reading. The score is derived from configured weights and the validation results.
- `imputed`: whether the published `value` was computed by an imputation strategy rather than taken directly from the calibrated/raw reading.
- `raw_value`: the original sensor value as read from hardware, preserved for auditability when the published `value` differs due to imputation or smoothing.
- `calibrated_value`: the sensor value after calibration is applied (before imputation/smoothing).
- `normalized_value`: the calibrated value after normalization is applied.
- `calibration_profile_id` / `normalization_profile_id`: identifiers of the profiles used to transform the reading, allowing downstream consumers to reproduce the same transformations.

3.2.4 Alert events

Alerts are expressed as append-only lifecycle events. Instead of mutating a single “alert row” in-place, the system records each important transition as a new event. This provides an audit trail and allows consumers to reconstruct the alert state at any point in time by replaying the events.

Alert events are transported on `dt.alerts` as JSON payloads and are represented by `AlertHistoryEvent` and its variants (`dt/communication/dataclasses/alerts/alert_record.py`). Each event includes:

- identifiers (`alert_key`, `plant_id`) so consumers can group and deduplicate,
- a timestamp (seconds since epoch),
- a status in `{active, ignored, acknowledged, cleared}`,
- a severity (`info`, `warning`, `critical`),
- a human-readable message,
- and the `correlation_id` of the originating cause (typically a sensor reading).

Two event subtypes carry additional context:

- `SensorAlertEvent` embeds the triggering `ProcessedSensorData` as `reading`, plus optional threshold/range parameters that explain **why** the alert was raised.
- `ExternalAlertEvent` embeds free-form `metadata` for alerts raised by external modules or operator-driven flows.

❏ Goal

Currently the `ExternalAlertEvent` dataclass is a place holder for future use cases. In the future, every alert “type” will need to have a corresponding event schema that captures the relevant context for that type. For example, an “equipment failure” alert might include fields for the affected equipment ID, error codes, and maintenance history.

The key point is that the `metadata` field is only a temporary catch-all for non-sensor alerts, and the long-term goal is to have well-defined schemas for each alert type that capture the necessary context for operators and downstream consumers.

Alert definitions (`AlertDefinition`) capture the **identity** of an alert: the properties that should remain stable across the alert’s lifetime and across repeated firings. They are keyed by `(alert_key, plant_id)` so the system can reason about “the same alert” even when it emits multiple history events over time. In practice, the definition stores invariants such as the originating source (topic name or external), the optional `sensor_id`, the `kind` (output type of the Alert, either Sensor, or External), the rule identifiers/names, and operational parameters like `persistence_count` and `cooldown_seconds`.

Keeping these invariants separate from `AlertHistoryEvent` has two benefits. First, lifecycle events stay small and focused on **what happened** (status transition, timestamp, message, `correlation_id`), while the definition answers **what this alert represents** without repeating the same metadata in every event. Second, it gives the database a stable handle for joins and queries: before publishing an event to Kafka, the alert engine ensures the corresponding definition exists via an idempotent upsert. That way, even after service restarts, consumers can replay the `dt.alerts` stream for state reconstruction and still rely on the database to resolve the “definition” of any alert key they encounter.

3.2.5 Queries (REST)

The database service exposes a read-oriented REST API that complements Kafka. REST **currently** is used for queries over historical data and current alert state. To keep contracts consistent, query parameters are represented as dataclasses (`dt/communication/dataclasses/queries/*.py`) and serialized into URL parameters.

The main query shapes are:

`ReadingsQuery` (database: `GET /readings`) retrieves event-level processed readings or hourly aggregates. The `window` parameter selects the shape:

- `window=raw` returns per-reading `ProcessedSensorData` records (event-level time series).
- `window=1h` returns `AggregatedReading` records computed by TimescaleDB continuous aggregates.

Filtering can be applied by `sensor_id`, `plant_id`, `topic`, and optional time bounds `since` and `until`.

`ActiveAlertsQuery` (database: `GET /alerts/active`) returns the currently active alert state as the latest lifecycle event per alert key.

`AlertHistoryQuery` (database: `GET /alerts/history`) returns alert lifecycle history for a given plant (optional) with a bounded `limit`.

3.2.6 Camera snapshots

Camera snapshots are treated as first-class events, but they are **not** represented as `RawSensorData` / `ProcessedSensorData`. The payload is a JPEG and needs image-specific metadata that does not fit the scalar sensor-reading model.

From a routing perspective, the snapshot travels through the same backbone as other processed signals:

- the collector emits snapshots on Kafka `dt.sensors.processed.camera_image`,
- the database service persists them in a dedicated `camera_snapshots` table,
- the webapp queries “latest snapshot” over REST for UI rendering.

The required fields are:

- `plant_id`: relational identifier of the plant being monitored.
- `sensor_id`: relational identifier of the camera sensor that produced the snapshot.
- `timestamp`: Unix epoch timestamp in **seconds** (float) when the picture was captured.
- `topic`: semantic base topic (`dt.sensors.camera_image`). Note that the Kafka topic used for transport is the processed variant `dt.sensors.processed.camera_image`.
- `correlation_id`: opaque trace identifier (UUID string) generated at capture time.
- `mime_type`: image MIME type (currently `image/jpeg`).
- `image`: base64-encoded JPEG bytes (ASCII string, without a `data:` prefix).
- `width` / `height`: optional integer dimensions in pixels when known by the producer.

The database service stores snapshots in a dedicated `camera_snapshots` table and exposes a read API to retrieve the latest snapshot per plant. The webapp proxies this endpoint and adapts the shape for browser usage:

- `/api/camera/snapshots/latest?plant_id=<int>`

3.2.7 Controller module (control contracts)

The controller module introduces the system’s “closed-loop” surface area: deciding **what to do** and ensuring actuator commands are executed safely and auditable.

At the contract level, the controller speaks in three distinct message families:

- **Mode**: who is allowed to control a plant right now (routine vs AI).
- **Routines**: how operators encode automation logic (a UI-authored graph compiled into rules).
- **Actions**: how commands are requested, executed, and logged.

These schemas live in `dt/communication/dataclasses/controller/` and are reused consistently across:

- controller service REST (`dt/controller/api.py`) for orchestration,
- database service REST (`dt/data/database/api.py`) for persistence and history,
- webapp REST proxy (`dt/webapp/api.py`) for browser-friendly access.

Kafka support exists for action events on `dt.actions`, but the controller currently does not publish to it.

3.2.7.1 Control mode (ControlMode)

`ControlMode` expresses “who owns actuation” for a plant. In practice, this contract is used as a runtime interlock: routine-driven actuation should be suppressed while AI auto-pilot is enabled.

It is exposed via:

- controller service: `GET /controller/mode?plant_id=<int>` and `PUT /controller/mode`
- webapp proxy: `GET /api/control/mode?plant_id=<int>` and `PUT /api/control/mode`
- database service: `GET /controller/mode?plant_id=<int>` and `PUT /controller/mode`

Schema (dt/communication/dataclasses/controller/routines.py):

- `plant_id` : plant identifier.
- `owner` : string in { "routine", "ai" } indicating which automation source is allowed to dispatch.
- `updated_at` : optional timestamp string from the persistence layer.

In the controller runtime, if `owner="ai"` it acts as an interlock that prevents routine actions from executing (they are logged as skipped with a reason).

3.2.7.2 Action dispatch (`ActionDispatch`)

The action path starts with an intent to act. `ActionDispatch` is the client payload used to request a manual or AI-triggered actuator action. The controller validates the request, assigns defaults (for example `duration=0.0`), and turns it into an `ActionCommand` for execution.

It is submitted to:

- controller service: `POST /controller/actions/dispatch`
- webapp proxy: `POST /api/actions/dispatch`

Schema (dt/communication/dataclasses/controller/action_dispatch.py):

- `plant_id` : plant identifier (**must be > 0**).
- `actuator_id` : actuator identifier (**must be > 0**).
- `command` : actuator command string (**must be non-empty**; for example "ON" / "OFF").
- `source` : string in { "manual", "ai" }.
- `duration` : duration in seconds (float, defaults to `0.0` , must be `>= 0`).
- `reason` : optional non-empty string describing why the action is requested.
- `action_id` : optional non-empty string to provide a stable logical identifier.
- `correlation_id` : optional non-empty trace identifier. If omitted, the controller service generates one.

3.2.7.3 Action command and history (`ActionCommand`)

`ActionCommand` is the canonical record of “an actuator action that is being executed (or was executed)”. This is the object the controller uses internally, and it is also the object that gets persisted and later returned to the UI as action history. Conceptually, `ActionDispatch` is an input request and `ActionCommand` is the normalized, auditable execution record.

Schema (dt/communication/dataclasses/controller/action_command.py):

- `plant_id` : plant identifier (**must be > 0**).
- `action_id` : non-empty string used to deduplicate and group actions (stable logical key).
- `actuator_id` : actuator identifier (**must be > 0**).
- `started_at` : Unix epoch timestamp in **seconds** (float) when execution started.
- `duration` : intended duration in seconds (float, must be `>= 0`).
- `command` : non-empty command string.
- `reason` : non-empty human-readable reason.
- `correlation_id` : non-empty trace identifier.
- `source` : string in { "manual", "ai", "routine" }.
- `routine_id` : optional routine identifier; **required** when `source="routine"` .
- `status` : optional string describing execution state persisted by the storage layer (`running` , `completed` , `failed` , `rejected` , `skipped`).
- `error_message` : optional error description when execution fails or is skipped/rejected.
- `ended_at` : optional Unix epoch timestamp in **seconds** (float) when execution ended.

Persistence and history:

- the database service accepts action execution upserts via `POST /actions/log` with body `{"action": <ActionCommand>}`. The `status` field is required when logging.
- clients read action history via `GET /controller/actions/history?plant_id=<int>&limit=<int>` or the webapp proxy `GET /api/actions/history?plant_id=<int>&limit=<int>`.

Kafka transport:

- the messaging layer supports consuming `ActionCommand` events from `dt.actions`,
- the controller currently does not publish to `dt.actions` by default (publishing is present as a commented call site in the actuator manager).

i Info

Instead of upserting the `ActionCommand` directly to the Database service, I will make the controller publish the `ActionCommand` to Kafka topic `dt.actions`, and then have the Database service consume from that topic and persist the records. This way, the webapp and any other interested consumers can also subscribe to `dt.actions` for real-time updates on actuator activity.

3.2.7.4 Routines (authoring + compiled representation)

Routines let operators define automation logic in the dashboard Logic Builder. The UI produces a graph (`RoutineGraph`) because that is the natural editing shape. The controller compiles that graph into flat rules (`CompiledRule`) because that is the natural execution shape.

Authoring graph schema (`dt/communication/dataclasses/controller/routines.py`):

- `RoutineGraph`
 - `nodes`: list of `RoutineNode` objects.
 - `edges`: list of `RoutineEdge` objects describing directed links between node ids.
 - `name`: routine name (must match `RoutineUpdate.name` when provided).
 - `plant_id`: plant id (must match `RoutineUpdate.plant_id` when provided).
 - `ui`: optional map of node id → `UiNode` containing UI-only layout metadata.
- `RoutineNode`
 - `id`: node id unique within a graph.
 - `kind`: "trigger" or "action".
 - `trigger`: present when `kind="trigger"`.
 - `action`: present when `kind="action"`.
- `RoutineEdge`
 - `source`: source node id.
 - `target`: target node id.
- `UiNode`
 - `x`, `y`: UI coordinates.
 - `label`: display label.

Compiled representation (`CompiledRule`, `Trigger`, `Action`):

- `CompiledRule.id`: stable rule identifier within a routine.
- `Trigger.type`: one of {"sensor", "time", "date", "interval"}.
 - sensor triggers use `topic`, `op` (comparison operator), and `value` (numeric threshold).
 - time triggers use `time` ("HH:MM").
 - date triggers use `date` ("YYYY-MM-DD") and optionally `time`; if `time` is omitted, the scheduler fires at midnight.

- interval triggers use `every_days` (positive integer) and `at` (`"HH:MM"`), anchored on the routine creation date.
- `Action`: `actuator_id`, `command`, `duration` (seconds).

Persisted routine schema:

- `Routine`: includes `id`, `plant_id`, `name`, `enabled`, `graph`, optional `compiled_rules`, and optional `created_at` / `updated_at` timestamp strings.
- `RoutineUpdate`: patch payload used for create/update (optional `plant_id`, `name`, `enabled`, `graph`, `compiled_rules`).

The controller service requires `plant_id`, `name`, and `graph` for routine creation, and will generate `compiled_rules` server-side by compiling `graph` before persisting.

3.3 Interfaces (REST + Socket.IO)

The system uses two complementary interface styles.

Kafka is the primary mechanism for streaming data and fan-out between services. REST and Socket.IO sit on top of that backbone and exist for two reasons: **(1)** querying state that is not naturally expressed as an event stream (historical windows, current active alerts, metadata), and **(2)** enabling interactive workflows between services (submitting, acknowledging, and clearing alerts, pushing live updates to browsers, setting up routines).

This section documents the service boundaries as they exist in the codebase today. The intent is operational: if a client times out, a dashboard view is empty, or an integration needs to be added, you should be able to find the authoritative endpoint, its input contract, and the expected response shape here.

3.3.1 Database service (REST)

The database service runs a Kafka bridge that persists processed readings, alert events and control settings, and it exposes a read-oriented REST API so other modules (and the webapp) can query historical data, metadata, alert state and routines. It listens on port `5001` (`dt/data/database/app.py`).

Most endpoints return JSON arrays of canonical dataclass payloads serialized with the `generic` adapter. The service uses **seconds** since epoch at the API boundary for timestamps and time filters.

Besides query endpoints, the database service also owns a small set of **write-side** endpoints for metadata registration and persistence of control state (modes, routines, and action execution records).

Endpoint	Method	Purpose and contract
<code>/bind_sensor</code>	POST	Register a sensor and obtain its <code>sensor_id</code> . Body: <code>SensorDescriptor</code> . Response: <code>{status, sensor_id}</code> . Returns <code>400</code> on invalid JSON/payload.
<code>/sensors</code>	GET	List registered sensors as <code>SensorDescriptor[]</code> .
<code>/readings</code>	GET	Query readings or aggregates. Query: <code>ReadingsQuery</code> (<code>window</code> in <code>{raw, 1h}</code> , optional <code>sensor_id</code> , <code>plant_id</code> , <code>topic</code> , <code>since</code> , <code>until</code>). Returns <code>ProcessedSensorData[]</code> when

		<code>window=raw</code> , or <code>AggregatedReading[]</code> when <code>window=1h</code> . Returns <code>400</code> on invalid query params.
<code>/camera/snapshots/latest</code>	GET	Return the latest camera snapshot for a plant. Query: <code>plant_id</code> (required). Returns <code>CameraSnapshot</code> on <code>200</code> , <code>404</code> when no snapshot exists.
<code>/actuators</code>	GET	List actuators (JSON objects returned by storage).
<code>/plants</code>	GET	List plants (JSON objects returned by storage).
<code>/bind_actuator</code>	POST	Register an actuator. Body: <code>JSON {plant_id, name, pin, relay_channel}</code> . Response: <code>{status, actuator_id}</code> . Returns <code>400</code> on invalid JSON/payload.
<code>/actions/log</code>	POST	Upsert an action execution record. Body: <code>{"action": <ActionCommand>}</code> ; <code>ActionCommand.status</code> must be present. Response: <code>{status:"ok"}</code> . Returns <code>400</code> on invalid JSON/payload.
<code>/controller/mode</code>	GET	Get current control mode. Query: <code>plant_id</code> (required). Returns <code>ControlMode</code> .
<code>/controller/mode</code>	PUT	Persist control mode. Body: <code>ControlMode</code> . Response: <code>{status:"ok"}</code> . Returns <code>400</code> on invalid JSON/payload.
<code>/controller/routines</code>	GET	List routines for a plant. Query: <code>plant_id</code> (required). Returns <code>Routine[]</code> .
<code>/controller/routines</code>	POST	Create a routine. Body: <code>RoutineUpdate</code> with <code>plant_id</code> , <code>name</code> , <code>graph</code> , and <code>compiled_rules</code> required at this boundary. Response: <code>{id, status}</code> with <code>201</code> . Returns <code>400</code> on invalid JSON/payload.
<code>/controller/routines/<routine_id></code>	PUT	Update a routine. Body: <code>RoutineUpdate</code> (patch). Response: <code>{status:"updated"}</code> . Returns <code>400</code> on invalid JSON/payload.
<code>/controller/routines/<routine_id></code>	DELETE	Delete a routine. Response: <code>{status:"deleted"}</code> .
<code>/controller/actions/history</code>	GET	Get action execution history. Query: <code>plant_id</code> (required), <code>limit</code> (default <code>50</code>). Returns <code>ActionCommand[]</code> .
<code>/alerts/definitions</code>	POST	Idempotent upsert of an <code>AlertDefinition</code> . Used by the alert engine before emitting events so the database can resolve alert

		identity consistently. Returns 400 on invalid JSON/payload.
/alerts/history	GET	Query alert lifecycle history. Query: <code>AlertHistoryQuery</code> (optional <code>plant_id</code> , <code>limit</code>). Returns <code>AlertHistoryEvent[]</code> (including sensor/external variants). Returns 400 on invalid query params.
/alerts/active	GET	Query currently active alerts. Query: <code>ActiveAlertsQuery</code> (optional <code>plant_id</code>). Returns <code>AlertHistoryEvent[]</code> representing the latest ACTIVE state per alert key. Returns 400 on invalid query params.

Two design choices are worth calling out. First, `/readings` supports two shapes via `window`: event-level processed readings (`raw`) and hourly aggregates (`1h`), so clients can choose between fidelity and cost. Second, alerts are queried from the database rather than from the alert engine because the database is the durable source of truth: it survives restarts and can provide a consistent “active alerts” view even when the engine is temporarily offline.

3.3.2 Alert engine (REST)

The alert engine exposes a small command-oriented REST API. It exists to let operators and other modules interact with alert lifecycle state (submit, acknowledge, clear) and to introspect the configured rule set. It listens on port 5003 (`dt/alerts/app.py`).

The API is intentionally minimal: listing active alerts and querying history are delegated to the database service, because those are durable query concerns. The alert engine’s in-memory registry is used for deduplication, persistence counters, and cooldown handling; on startup, it attempts to restore state by fetching active alerts from the database.

Endpoint	Method	Purpose and contract
/alerts/submit	POST	Submit an external alert and (if not suppressed by persistence/cooldown) publish an event to <code>dt.alerts</code> . Required JSON fields: <code>alert_key</code> , <code>plant_id</code> , <code>severity</code> , <code>message</code> , <code>correlation_id</code> . Optional: <code>metadata</code> , <code>source</code> , <code>sensor_id</code> , <code>persistence_count</code> , <code>cooldown_seconds</code> . Returns 202 with <code>{alert_key, status}</code> and 400 on validation errors.
/alerts/<alert_key>/acknowledge	POST	Acknowledge an active alert. JSON body requires <code>actor</code> . Returns 200 on success, 400 on missing actor, 404 if the alert is not known to the registry. Publishes an ACKNOWLEDGED event to <code>dt.alerts</code> .
/alerts/<alert_key>/clear	POST	Clear an active alert. Returns 200 on success, 404 if the alert is not known to the registry. Publishes a CLEARED event to <code>dt.alerts</code> .

<code>/alert-rules</code>	GET	List configured alert rules as JSON (serialized <code>AlertRule[]</code>). This endpoint is served on a separate blueprint (not under <code>/alerts</code>).
---------------------------	-----	--

Operationally, think of these endpoints as “write-side” actions: they change alert state and emit lifecycle events. For “read-side” views (dashboards, audits, debugging), use the database service endpoints `/alerts/active` and `/alerts/history`.

3.3.3 Controller service (REST)

The controller service is the command-oriented entry point for automation: it validates and compiles routines, enforces runtime ownership rules (routine vs AI), executes actuator commands, and persists the resulting state and history via the database service.

In practice, the control plane is a multi-hop flow:

- the browser calls the webapp under `/api`,
- the webapp forwards control requests to the controller service,
- the controller service persists durable state via the database service and executes actuator drivers locally.

The controller service listens on port `5004` (`dt/controller/app.py`) and exposes its API under the `/controller/*` prefix (`dt/controller/api.py`).

Endpoint	Method	Purpose and contract
<code>/controller/mode</code>	GET	Fetch current <code>ControlMode</code> . Query: <code>plant_id</code> (required). Returns <code>400</code> when <code>plant_id</code> is missing.
<code>/controller/mode</code>	PUT	Update <code>ControlMode</code> . Body: <code>ControlMode</code> . Response: <code>{status:"updated", mode:<ControlMode>}</code> on <code>200</code> . Returns <code>400</code> on invalid JSON/payload.
<code>/controller/routines</code>	GET	List routines for a plant. Query: <code>plant_id</code> (required). Returns <code>Routine[]</code> .
<code>/controller/routines</code>	POST	Create a routine. Body: <code>RoutineUpdate</code> with <code>plant_id</code> , <code>name</code> , <code>graph</code> required. The controller compiles <code>graph</code> server-side and persists <code>compiled_rules</code> . Returns <code>201</code> with <code>{id, status}</code> . Returns <code>400</code> if the graph is invalid.
<code>/controller/routines/<routine_id></code>	PUT	Update a routine. Body: <code>RoutineUpdate</code> (patch). If a new <code>graph</code> is provided, the controller recompiles it before persisting. Returns <code>{status:"updated"}</code> on <code>200</code> , or <code>400</code> on invalid graph.
<code>/controller/routines/<routine_id></code>	DELETE	Delete a routine. Optional query: <code>plant_id</code> (used as a refresh hint). Returns <code>{status:"deleted"}</code> .

/controller/actions/dispatch	POST	Dispatch a manual or AI action. Body: ActionDispatch. Returns {"status": "accepted", "action_id": <str>} on success, or 400 on validation errors.
/controller/actions/history	GET	Fetch action execution history. Query: plant_id (required), limit (default 50). Returns ActionCommand[].

3.3.4 Webapp (REST)

The webapp exposes a small REST API under /api that is tailored to browser needs. It is effectively a proxy over the database service: it forwards queries, converts time units and field names for plotting, and returns JSON in the shape the front-end expects. It listens on port 5000 (dt/webapp/app.py).

The webapp is also the browser-facing gateway for control actions. For read-heavy concerns (readings, alerts, metadata, snapshots), it talks to the database service. For command-oriented concerns (mode, routines, dispatch), it talks to the controller service and keeps the boundary consistent for the front-end.

The key difference from the database service API is time representation. Browser clients use milliseconds since epoch, so the webapp accepts since / until in **milliseconds**, forwards them to the database in **seconds**, and returns time in milliseconds (renaming timestamp or bucket accordingly). Control endpoints do not apply timestamp unit conversions, they forward JSON payloads as-is.

Endpoint	Method	Purpose and contract
/api/readings	GET	Browser-friendly reading query. Same filters as ReadingsQuery, but since/until are in ms and responses use time in ms. Returns processed readings (window=raw) or aggregates (window=1h).
/api/alerts/active	GET	Browser-friendly active alerts list, with timestamps mapped to time in ms.
/api/alerts/history	GET	Browser-friendly alert history list, with timestamps mapped to time in ms.
/api/camera/snapshots/latest	GET	Fetch the latest camera snapshot. Query: plant_id (optional; defaults to 1). Returns 200 with a browser-shaped payload (time in ms, topic omitted), or 404 when no snapshot exists.
/api/sensors	GET	List sensors as SensorDescriptor[].
/api/actuators	GET	List actuators (proxied from the database service).
/api/control/mode	GET	Get current ControlMode. Query: plant_id (required).
/api/control/mode	PUT	Set ControlMode. Body: JSON {plant_id, ai_autopilot_enabled, owner}. Returns the updated ControlMode.

/api/routines	GET	List routines for a plant. Query: <code>plant_id</code> (required). Returns <code>Routine[]</code> .
/api/routines	POST	Create a routine. Body: <code>RoutineUpdate</code> with <code>plant_id</code> , <code>name</code> , <code>graph</code> required. Returns <code>201</code> with <code>{id, status}</code> .
/api/routines/<routine_id>	PUT	Update a routine. Body: <code>RoutineUpdate</code> (patch). Returns <code>{status:"updated"}</code> .
/api/routines/<routine_id>	DELETE	Delete a routine. Returns <code>{status:"deleted"}</code> .
/api/actions/dispatch	POST	Dispatch a manual or AI action. Body: <code>ActionDispatch</code> . Returns <code>{status, action_id}</code> on success.
/api/actions/history	GET	Fetch action execution history. Query: <code>plant_id</code> (optional; defaults to <code>1</code>), <code>limit</code> (default <code>50</code>). Returns <code>ActionCommand[]</code> .

3.3.5 Webapp (Socket.IO)

Socket.IO is the webapp's real-time interface. It is fed by a Kafka-to-SocketIO bridge (`dt/webapp/consumer.py`) that subscribes to processed sensor topics and to `dt.alerts`, caches the latest payloads in memory, and emits updates to connected browsers.

There are three families of Socket.IO events:

Connection status. When a client connects, the server emits `connection_status` with `{connected: bool}`. The intent is UX: the dashboard can show “disconnected” when Kafka is down even if the Flask app itself is reachable.

Processed sensor readings. For each sensor type, the bridge emits on an event name equal to the Kafka processed topic string: `dt.sensors.processed.<type>`. The payload is a browser-shaped `ProcessedSensorData` representation: `timestamp` is converted to `time` in milliseconds, and the `topic` field is removed because the event name already carries the type information.

Alert lifecycle updates. Alerts are emitted as two events:

- `alerts_update` carries an alert event payload (browser-shaped `AlertHistoryEvent`) plus an additional `alert_id` field formed as `<plant_id>:<alert_key>`, which is used as a stable cache key by the UI.
- `alerts_remove` carries only the `alert_id` string and indicates a cleared alert should be removed.
- **Camera snapshots:** Camera images are emitted on the dedicated event name `dt.sensors.processed.camera_image`. It is a separate event family because the payload is a base64-encoded JPEG string rather than a JSON object, and the UI needs to handle it differently.

On connect, the server also replays its caches (latest readings per topic and current active alerts) so a freshly opened dashboard does not need to wait for the next Kafka message to render a meaningful state.

4 Modules

This section describes the codebase in terms of its top-level Python packages under `dt/`. In practice, most packages correspond to a runnable service (collector, preprocessing, database, webapp, alerts), while a few are shared libraries that provide contracts, configuration, and common utilities.

Each subsection below aims to answer the same operational questions: what the package is responsible for, which interfaces it relies on (Kafka topics and/or HTTP endpoints), what it emits, which configuration knobs matter in day-to-day operation, how to run it locally, and what to look at first when something goes wrong. The detailed configuration model is centralized in the following section, so the configuration paragraphs below only keep the minimum needed to understand what each module depends on and where to continue reading.

4.1 `dt/communication`

The `dt/communication` package is the interoperability layer of the system. It defines the topic names, the dataclasses that represent on-wire payloads, and the serialization adapters that make those dataclasses usable across contexts (Kafka JSON messages, database rows, and Spark rows/tuples). Most other modules depend on it, and changes here tend to have cross-service impact.

4.1.1 Purpose

At a high level, `dt/communication` provides a single source of truth for the event contracts used in the pipeline: raw/processed sensor readings, alert lifecycle events, actuator commands and routine setting. The package deliberately stays free of runtime side effects (see `dt/communication/__init__.py`) so it can be imported from any service without triggering connections or other operational concerns. It is purely a library of schemas.

i Info

This package is not a standalone service and does not expose an endpoint on its own. Instead, it defines the contracts that other services consume and produce.

! Warning

Because this package is central and entirely shared and written in pure Python, if one module were to be entirely rewritten in another language (for example Rust or Go), we would need to re-implement the dataclasses and adapters in that language, and maintain the contract manually across the two implementations by ensure that the schemas respect the one defined in this runbook.

4.1.2 Configuration

`dt/communication` relies on the shared configuration enum in `dt/utils/config.py` indirectly through clients that need network endpoints. In particular, `DatabaseApiClient` defaults to `Config.FLASK_DB_URL` for the database service base URL.

4.1.3 Components

The package is organized around a few core components:

Dataclasses. Under `dt/communication/dataclasses/`, the main payloads are `RawSensorData` and `ProcessedSensorData` (plus `ValidationFlag`). Supporting DTOs cover sensor meta-

data (`SensorDescriptor`), time-bucketed aggregates (`AggregatedReading`), actuator commands (`ActionCommand`), and alert lifecycle events (`AlertDefinition`, `AlertHistoryEvent`, `SensorAlertEvent`, `ExternalAlertEvent`). It also contains the dataclasses schema used on the REST API endpoints of the different services `ActiveAlertsQuery`, `ReadingsQuery` as well as the contract associated to the controller module `Routine`, `RoutineUpdate`, `ActionCommand`.

Adapters. Under `dt/communication/adapters/`, the `dump(...)` and `load(...)` functions route to format-specific adapters: `generic` for JSON-safe Python objects (Kafka and REST), `db_row` for PostgreSQL/TimescaleDB persistence, `spark_row` for PySpark `Row` objects, and `tuple` for Spark state storage.

Messaging service. `KafkaService` (`dt/communication/messaging_service.py`) implements the `MessagingService` interface on top of `kafka-python`. It publishes JSON payloads and dispatches subscribed messages to callbacks.

Database API client. `DatabaseApiClient` (`dt/communication/db_client.py`) is the thin HTTP client used by other services to interact with the database service (binding sensors, querying readings, and retrieving or upserting alert definitions and history).

Controller API client. `ControllerApiClient` (`dt/communication/controller_client.py`) is a similarly thin HTTP client for the controller service. It will be used by the webapp to send actuator commands and setup routines based on threshold breaches, date, time, or other conditions.

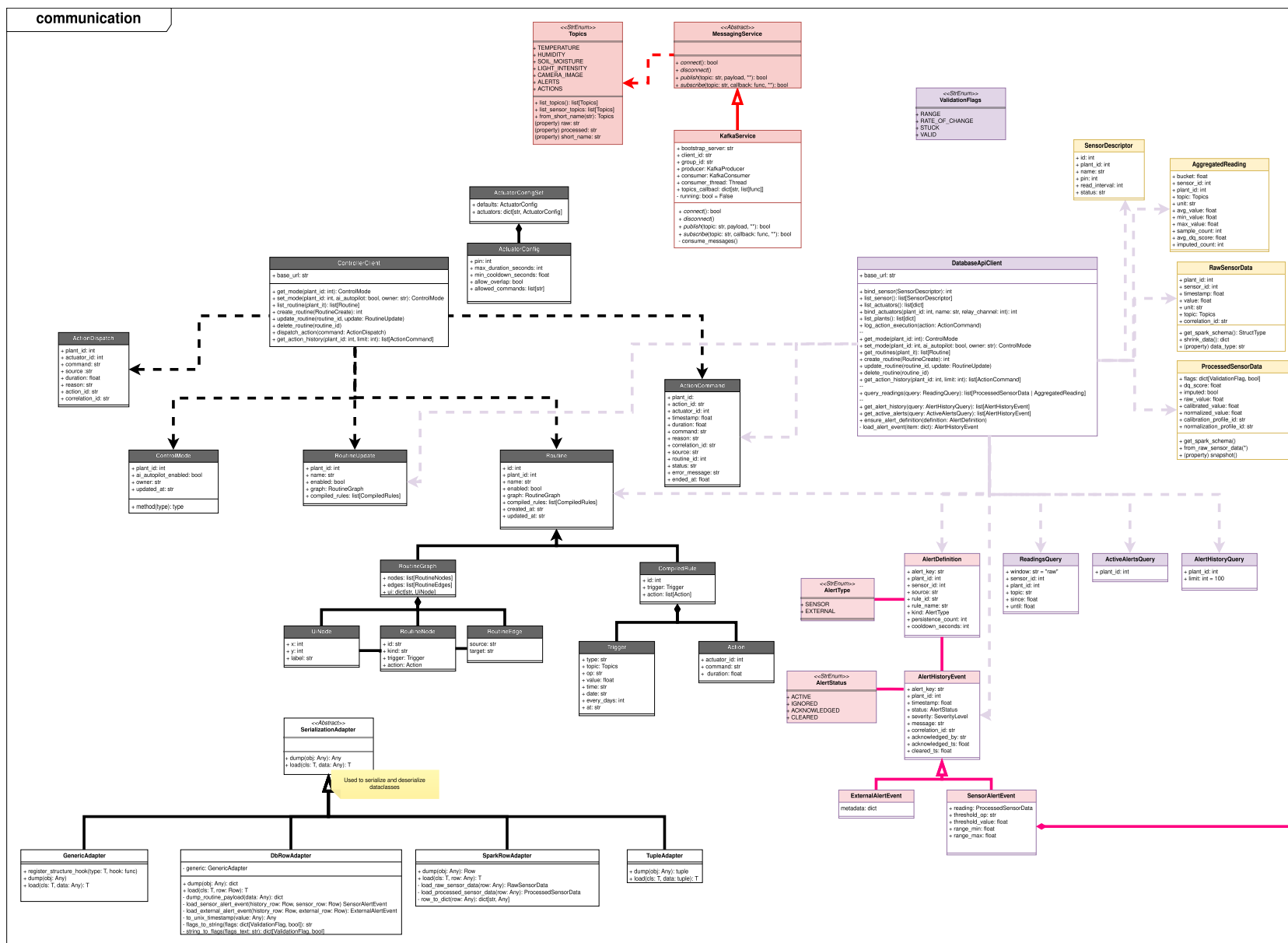


Figure 8 : Class diagram of the `dt/communication` package, showing the dataclasses, adapters, and clients.

4.2 dt/collector

The `dt/collector` package is the hardware-facing entry point of the system. It polls sensors connected to the Raspberry Pi, converts each reading into a `RawSensorData` payload, and publishes it to Kafka on a fixed schedule. It also registers sensors with the database service so downstream services can resolve `sensor_id` values to stable names and configuration keys.

4.2.1 Purpose

`dt/collector` is intentionally simple: it runs an infinite loop, reads each sensor at its configured interval, and emits time-stamped events. It does not perform calibration, validation, imputation, or smoothing. Those steps belong to the preprocessing pipeline.

4.2.2 Inputs and outputs

Inputs are physical sensor devices (GPIO for the DHT22, and I²C for the STEMMA soil sensor, the BH1750 light sensor and the Camera) plus two network dependencies: **Kafka** (to publish events) and the **database service** (to bind sensors).

On startup, each sensor is converted to a `SensorDescriptor` and registered via `POST /bind_sensor` through `DatabaseApiClient`. The returned `sensor_id` is then attached to every emitted `RawSensorData` message.

Each reading is published by `SensorManager` (`dt/collector/sensor_manager.py`) using `KafkaService`. The intended output topic family for these messages is `dt.sensors.raw.*` so that the preprocessing job can consume them and publish cleaned readings to `dt.sensors.processed.*`.

4.2.3 Configuration

Most operational configuration is implicit in how the collector is wired: sensor names and pins are currently instantiated in code (`dt/collector/main.py`), and sensor timing is driven by each sensor's `read_interval` (seconds).

For network dependencies, the collector reaches the database service through `DatabaseApiClient`, which defaults to `Config.FLASK_DB_URL`, and it publishes events to Kafka through `KafkaService`.

4.2.4 Components

`Sensor` (`dt/collector/kinds/base_sensor.py`) defines the shared behaviour of a polling sensor: interval scheduling (`needs_data(...)`), a `read()` method that builds `RawSensorData`, and `to_dataclass()` for database binding. Concrete implementations encapsulate hardware access:

- `TemperatureSensor` and `HumiditySensor` read from a DHT22 via a shared
- `DHT22Singleton`, `SoilMoistureSensor` reads the Adafruit STEMMA soil sensor via
- `Seesaw` on the I²C bus and `LightSensor` reads lux from the BH1750 via I²C.
- `CameraSensor` captures images from the Raspberry Pi camera and encodes them as base64 strings

`SensorManager` is the orchestrator: it owns the sensor registry, binds sensors to the database, and publishes each due reading to Kafka. Its `seconds_until_next_read()` method supports a single-threaded loop that sleeps until the next sensor is due, which keeps the collector CPU usage low while still respecting per-sensor intervals.

4.2.5 Run command

Run the collector via the Makefile target: `make run-collector` which executes `dt/collector/main.py`. On a Raspberry Pi, ensure the required hardware dependencies are installed (Poetry `rpi` group) and that I²C is enabled for the I²C sensors.

4.2.6 Failure modes and troubleshooting

If sensor binding fails, `SensorManager.bind_sensor()` raises `BadSensorBindingException`. This typically indicates that the database service is not reachable at `FLASK_DB_URL` or returned an error for the bind request.

If Kafka is unreachable, `KafkaService.connect()` logs an error and publishing will fail, the collector will keep running but no downstream service will receive updates. If you see the collector running but no data in the dashboard, confirm Kafka connectivity and verify the topic family in use (`dt.sensors.raw.*` vs `dt.sensors.processed.*`).

Finally, note that hardware reads can be noisy. The DHT22 drivers frequently raise `RuntimeError` during reads, the current implementation logs the error and returns `-1`, which then propagates as the emitted `value`. Downstream services should treat such points as invalid and handle them through validation and imputation.

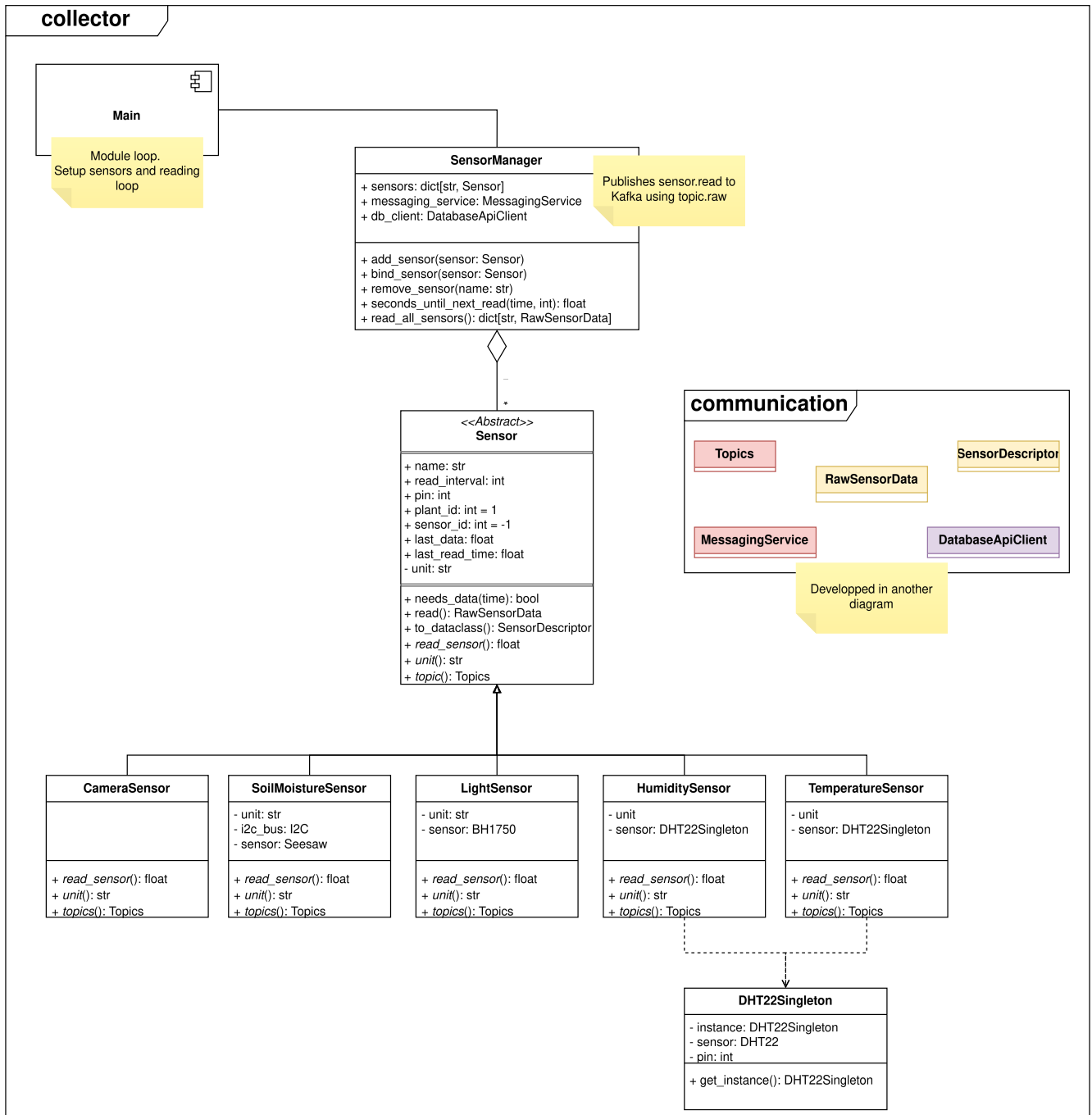


Figure 9 : Class diagram of the `dt/collector` package, showing the sensor classes, the `SensorManager` , and the interactions with Kafka and the database service.

4.3 dt/data/preprocess

The `dt/data/preprocess` package is the data-quality gate between the hardware collector and the rest of the platform. It runs as a Spark Structured Streaming job: it consumes raw sensor events, applies a deterministic preprocessing pipeline (calibration → validation → imputation → smoothing → normalization), and publishes `ProcessedSensorData` events that are safe to persist, visualize, and use for alerting and analytics.

Operationally, this service is where “messy edge telemetry” becomes a stable stream with explicit quality indicators. Every emitted point carries validation flags and a `dq_score`, and keeps traceability fields (`raw_value`, `calibrated_value`, `calibration_profile_id`, `normalization_profile_id`) so you can explain how and why the final `value` was produced.

4.3.1 Purpose

The preprocessing job is responsible for two things: ensuring that downstream modules never have to guess whether a value is trustworthy, and enforcing a consistent, configurable interpretation of the raw signals.

The pipeline itself is implemented as a Chain of Responsibility (`ProcessingPipeline` in `dt/data/preprocess/core/pipeline.py`). Each processor mutates a shared `ProcessingContext` object:

1. calibration standardizes the raw reading
2. validation evaluates range / rate-of-change / stuck checks and computes the DQ score
3. imputation attempts to recover invalid points using history
4. smoothing optionally reduces noise,
5. normalization maps calibrated values into a standard range when needed.

Because several checks and imputation strategies depend on historical context, the job is stateful. State is maintained per (`plant_id`, `sensor_id`) group using Spark’s `applyInPandasWithState` and a `SparkStateProvider` (`dt/data/preprocess/core/state.py`) that persists the latest accepted reading, flatline metadata, and a rolling history window.

4.3.2 Inputs and outputs

Input is the raw sensor event stream published by the collector on the Kafka topic family `dt.sensors.raw.*`. The job subscribes via a regex pattern and parses each Kafka message into the `RawSensorData` schema (`dt/communication/dataclasses/raw_sensor_data.py`).

Output is a stream of `ProcessedSensorData` events published to `dt.sensors.processed.*`. The output Kafka topic encodes the “processed” stage, but the payload’s `topic` field intentionally keeps the base topic (for example `dt.sensors.temperature`) so downstream systems can consistently identify the signal while still routing by processing stage at the topic level.

4.3.3 Configuration

This service is configured in two layers. Environment variables define how the Spark job reaches Kafka, the database service, and its own runtime limits, while `PREPROCESSING_CONFIG_PATH` points to the YAML file that defines how each sensor is calibrated, validated, imputed, smoothed, and normalized. The YAML is resolved according to the sensor registry from the database, so the naming of sensors matters as much as the configuration. For the full structure, supported strategies, and operational guidance, see **the dedicated preprocessing configuration section**.

4.3.4 Components

The preprocessing package is organized around a small core that is independent of Spark, and a thin Spark integration layer.

The core pipeline lives under `dt/data/preprocess/core/`.

- `ProcessingContext` (`dt/data/preprocess/core/context.py`) is the mutable record that carries a reading through every stage and accumulates outputs such as flags, DQ score, and traceability fields.
- `ProcessingPipeline` and `PipelineBuilder` (`dt/data/preprocess/core/pipeline.py`) implement the stage sequencing as a Chain of Responsibility.
- `StateProvider` and `SparkStateProvider` (`dt/data/preprocess/core/state.py`) abstract the “latest valid + rolling history” concept so that validators and imputation strategies can share consistent history access.

The stage implementations live under `dt/data/preprocess/stages/`. Each stage has a processor (for example `ValidationProcessor`) that reads configuration, consults state where needed, and writes its result back into the context. The actual algorithms are intentionally small and plugable: calibration, imputation, smoothing, and normalization use strategy objects instantiated from the YAML config, while validation is expressed as explicit checks plus a deterministic scoring function.

Configuration is handled by `dt/data/preprocess/config/`. The `SystemConfig` and `SensorConfig` dataclasses in `dt/data/preprocess/config/types.py` define the supported YAML schema. `ConfigurationManager` (`dt/data/preprocess/config/manager.py`) loads the YAML, merges templates with overrides, builds a DB-backed sensor registry, and caches resolved strategies so processors can request them by sensor key without re-parsing configuration.

Spark-specific wiring is isolated in `dt/data/preprocess/spark_adapter.py`. It is responsible for watermarking, grouping by (`plant_id`, `sensor_id`), bridging Spark state to `SensorState`, and translating microbatches into `ProcessedSensorData` rows.

Finally, `dt/data/preprocess/main.py` is the service entrypoint: it reads Kafka `dt.sensors.raw.*`, invokes the adapter, and writes the processed stream back to Kafka under `dt.sensors.processed.*`.

4.3.5 Spark specificities

This service uses Spark Structured Streaming, which can be understood as “Spark jobs that run forever”. You still write `DataFrame` transformations, but Spark executes them continuously as new data arrives. Under the hood, Spark processes the stream in small **micro-batches**: each micro-batch is a short-lived batch job that takes the next chunk of Kafka events, runs the pipeline, and commits its progress before waiting for more data.

Two concepts matter immediately when you process sensor telemetry: **event time** and **late data**. Event time is the timestamp carried inside the message (here: `RawSensorData.timestamp`, an epoch timestamp produced by the collector). Late data is anything that arrives out of order (network delays, retries, stalled producers, broker hiccups). If you ignore ordering, stateful checks like rate-of-change and imputation can become inconsistent because “the past arrives after the future”.

Spark’s answer is watermarks and state. In our implementation, the Spark adapter creates an `event_time` column from the epoch seconds and applies an event-time watermark (`withWatermark`) before any stateful work. The watermark is a moving threshold that tells Spark how far behind it is willing to accept late events. Events older than the watermark are considered “too late” for stateful operators. We also propagate the current watermark into the pipeline and mark late events explicitly (`ValidationProcessor._check_late_event`) so the output payload can reflect what happened and so we avoid updating state based on late points.

State in Structured Streaming is always scoped to a key. We group the stream by `(plant_id, sensor_id)` and then process each group with `applyInPandasWithState`. Spark provides a `GroupState` handle for each key. You can think of `GroupState` as a per-key key-value store managed by Spark's state store: it persists across micro-batches, can be timed out, and is checkpointed so that the job can resume after a crash without “forgetting history”.

We do not expose `GroupState` directly to the domain code because it is a Spark API and it would leak streaming concerns into every processor. Instead, we use three layers:

First, `GroupState` is the Spark-owned persistence mechanism and the source of two streaming-time signals: `getCurrentWatermarkMs()` (the current watermark) and the timeout lifecycle (via `hasTimedOut` and `setTimeoutTimestamp`).

Second, `SparkStateProvider` is the adapter that turns `GroupState` into a domain-shaped interface (`StateProvider`). Internally it hydrates a `SensorState` value object (latest accepted reading, flatline metadata, rolling history), and every call that mutates state (`update`, `record_flatline`, `get_recent_history`) re-synchronises back into Spark by calling `group_state.update(...)`. This keeps the validation and imputation logic independent of Spark while still using the Spark state store as the single source of truth at runtime.

Third, `ProcessingContext` is per-reading working memory. For each incoming `RawSensorData`, we create a fresh context that references the shared `StateProvider` for that sensor group and carries stage outputs (calibrated reading, flags, DQ score, imputed value, normalized value). The pipeline is purely “context in, context out”: it does not talk to Spark, it only reads and writes the context and consults `StateProvider` when it needs history.

Finally, there is an operational implication: state needs a lifecycle. Spark will keep per-key state forever unless you configure timeouts. We set an event-time timeout based on the latest processed timestamp for the key, and when Spark reports a timeout we remove the state for that sensor group. This prevents the state store from growing unbounded when sensors disappear. The trade-off is that after a long downtime, the first few readings have little or no history, so rate-of-change checks and history-based imputations may behave differently until enough valid points have been accumulated again.

4.3.6 Run command

Run the preprocessing job via: `make run-preprocessing` which executes `dt/data/preprocess/main.py`. This requires the optional Spark dependencies to be installed (Poetry `spark` group).

4.3.7 Failure modes and troubleshooting

If the job appears to run but produces no processed output, start by checking configuration resolution. When the database service is unreachable, the configuration manager cannot build the sensor registry and the job will then log warnings and skip all readings as “unknown sensor”. Confirm `FLASK_DB_URL` is correct and that the database service is running and can answer `GET /sensors`.

If you see “unknown sensor” warnings for specific sensors, it typically means the sensor was not bound (so `sensor_id` is missing from the registry) or the stored sensor name does not match a key in `preprocessing_config.yml`. Bind the sensor through the collector/database flow and ensure the YAML has a matching entry (directly or via the generic `<type>.<metric>` fallback).

If many readings are emitted with `dq_score = 0.0` and `valid_data_point = false`, inspect the Spark logs for `DropReadingException` messages. This indicates that a reading failed validation and

the configured imputation strategy could not recover it (for example because there is insufficient history or the gap is too large). In those cases, tune the imputation window/max-gap parameters or increase the state history length.

Finally, be aware that state is time-bounded: the job sets an event-time timeout so that a sensor group's state is removed after prolonged inactivity. After a state eviction (for example when a sensor has been offline), early readings may behave differently because rate-of-change checks and imputation have no history until enough valid points are re-accumulated.

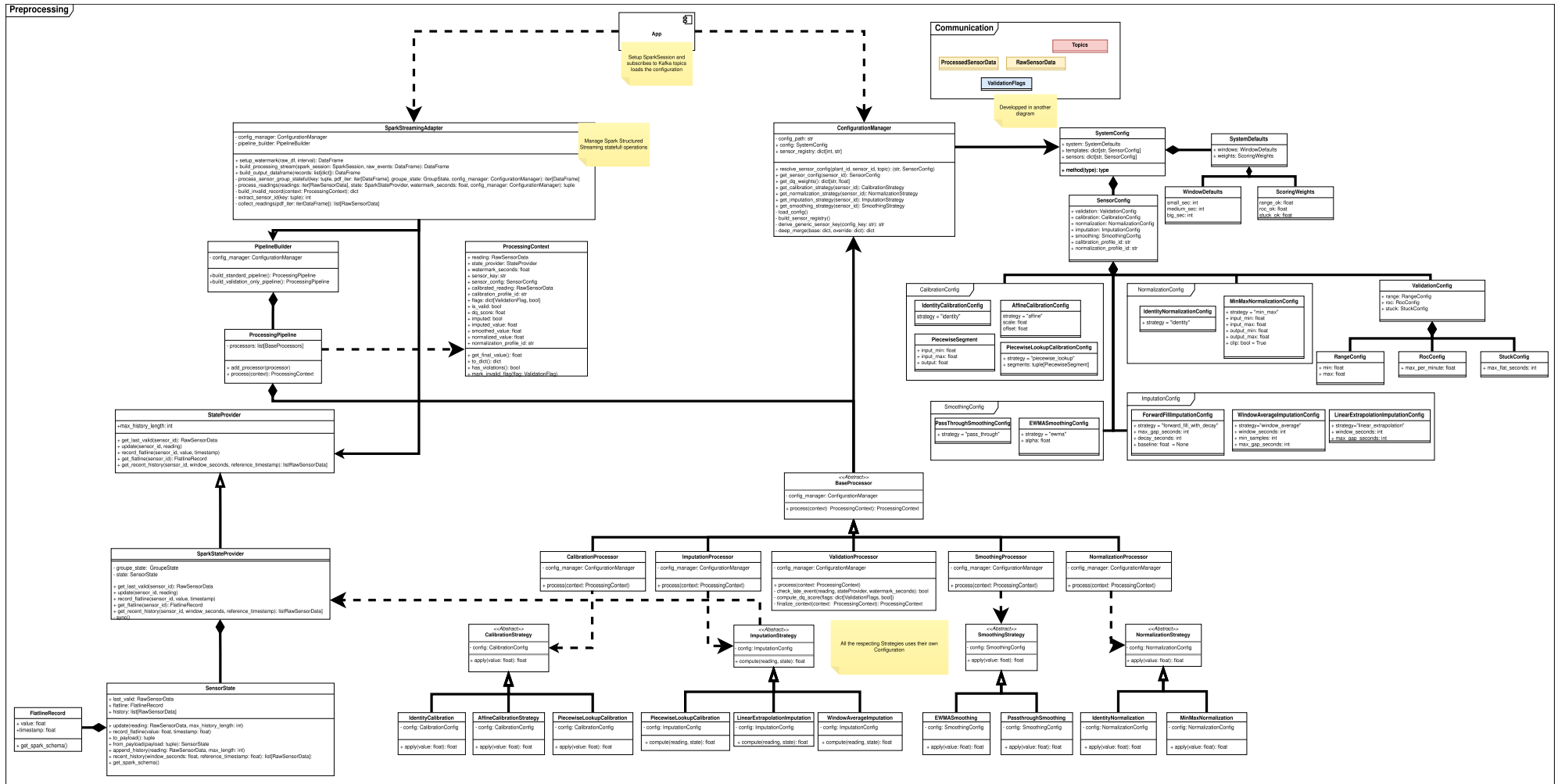


Figure 10 : Class diagram of the `dt/data/preprocess` package, showing the core pipeline components, the configuration manager, and the Spark adapter.

4.4 dt/alerts (future dt/analytics)

The `dt/alerts` package implements the alert engine service. It consumes processed sensor events, evaluates YAML-defined rules, maintains an in-memory state machine to avoid alert storms, and publishes alert lifecycle events to Kafka. It also exposes a small REST API for external alerts and for operator actions such as acknowledging and clearing alerts.

In the system taxonomy, alerting is part of the analytics layer. In this repository, it lives as its own package and runnable service under `dt/alerts`.

4.4.1 Purpose

The alert engine answers one question: given a processed sensor reading, should the system emit an alert event? Rule evaluation is stateless, but alert emission is intentionally stateful: `AlertRegistry` enforces two fatigue controls so that short spikes do not create noisy alert storms. `persistence_count` requires N consecutive matches before emitting, and `cooldown_seconds` prevents re-firing the same alert too often.

4.4.2 Inputs and outputs

Input is the processed sensor event stream `dt.sensors.processed.*`. The engine subscribes to every processed sensor topic via `Topics.list_sensor_topics()`.

The main output is the alerts topic `dt.alerts` (`Topics.ALERTS`). Every emitted event is an `AlertHistoryEvent` dataclass (either a `SensorAlertEvent` with the triggering reading snapshot and threshold context, or an `ExternalAlertEvent` with free-form metadata for now). Downstream consumers (database service, webapp) treat `dt.alerts` as the append-only audit log of alert lifecycle changes (to track active alerts, acknowledgments, and clearings), while the alert engine itself uses it as a source of truth for currently active alerts (hydrating its `AlertRegistry` at startup from `GET /alerts/active`).

i Info

The `ExternalAlertEvent` doesn't have specific type coercion or validation logic at this point. It is just a placeholder for future external alert submission (from other module). In the future all the alerts that we want to emit must be defined with a specific Dataclass and validation logic, so we can ensure the data quality and consistency of the alert events.

In addition to Kafka, `dt/alerts` depends on the database service REST API for two reasons: it hydrates its in-memory registry at startup from `GET /alerts/active`, and it ensures alert definitions exist by calling `POST /alerts/definitions` before publishing alert events.

4.4.3 Configuration

`KAFKA_URL` and `FLASK_DB_URL` let the service consume processed data, hydrate alert state (via `GET /alerts/active`), and persist alert definitions (via `POST /alerts/definitions`). Alerting rules are defined in a YAML file (default: `dt/utils/alert_rules.yml`, overridden via `ALERT_RULES_PATH`). The alert engine loads this file once. Therefore updating rules requires restarting the alert engine process to apply changes. The complete rule format and the meaning of persistence and cooldown are documented in **the alert configuration section**.

4.4.4 Components

The service is composed of a small set of components:

- **Rule loading.** `build_alert_rule_manager(...)` reads and validates the rules file.

- **Evaluation.** `RuleEvaluator` matches rules against each `ProcessedSensorData` and returns `(AlertDefinition, SensorAlertEvent)` candidates.
- **State.** `AlertRegistry` tracks alert state in memory (occurrence counts, cooldown, acknowledgment, clearing).
- **Publishing.** `AlertPublisher` persists the `AlertDefinition`, sets `acknowledged_ts` and `cleared_ts` when needed, then publishes to Kafka.
- **Kafka consumer.** `AlertEngineService` wires Kafka subscriptions to the evaluation and publishing path.
- **HTTP API.** `dt/alerts/api.py` exposes the REST endpoints for external alert submission and lifecycle actions.

4.4.5 REST API surface

The alert engine provides:

- `POST /alerts/submit` to submit an external alert.
- `POST /alerts/<alert_key>/acknowledge` with an `actor`.
- `POST /alerts/<alert_key>/clear`.
- `GET /alert-rules` to inspect the effective loaded rules.

4.4.6 Run command

Run the alert engine via: `make run-alert-engine`

For local development where Kafka is unavailable, the Makefile also provides an API-only mode: `make run-alert-api-only`

4.4.7 Failure modes and troubleshooting

If Kafka is down, `create_app()` fails fast so the service does not start in a half-working state.

If the database service is unavailable at startup, the engine logs a warning and continues without hydration, it will still evaluate rules and publish to Kafka, but it will not restore alert state across restarts.

If persisting an alert definition fails (`POST /alerts/definitions`), publishing is aborted: the service raises rather than emitting an event that would later fail persistence.

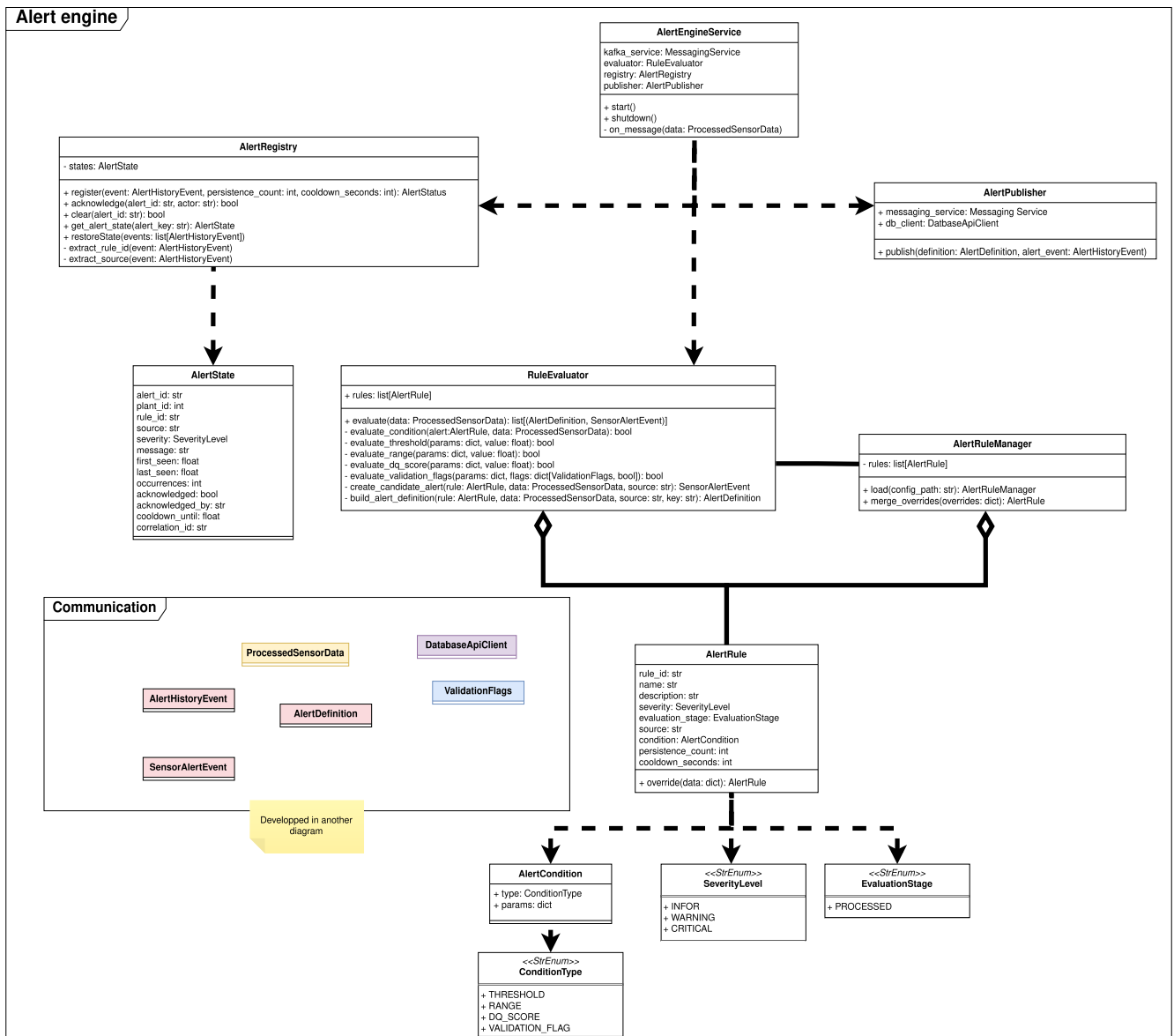


Figure 11 : Class diagram of the `dt/alerts` package, showing the main components: rule manager, evaluator, registry, publisher.

⚠ Warning

The alert engine is the first version of the future analytics module. In the future, we want to expand analytics responsibilities beyond alerting to include predictive insights, state evaluation, and other value-added processing on top of the raw and processed sensor data.

4.5 dt/data/database

The `dt/data/database` package implements the database service: a Flask REST API fronting a PostgreSQL + TimescaleDB backend, plus a Kafka-to-database bridge that persists streaming data. It is the system's source of truth for both time-series readings (hypertables and continuous aggregates) and relational domain data (sensors, plants, actuators, and alert lifecycle history).

4.5.1 Purpose

The database service plays two roles:

First, it persists data from Kafka. The bridge consumes processed sensor readings from `dt.sensors.processed.*` and alert lifecycle events from `dt.alerts`, and writes them to the TimescaleDB/relational tables.

Second, it serves data to other services and the dashboard through a stable HTTP interface. This keeps consumers simple: they can query a REST endpoint instead of embedding SQL or directly connecting to PostgreSQL.

4.5.2 Inputs and outputs

Kafka inputs are:

- `dt.sensors.processed.*` (as `ProcessedSensorData`) for measurements.
- `dt.alerts` (as `AlertHistoryEvent`) for alert history.

Outputs are:

- Persisted time-series and relational data in PostgreSQL/TimescaleDB.
- A REST API on port `5001` for binding sensors and querying readings/alerts.

4.5.3 Configuration

The database service is configured entirely through environment variables. The critical ones are `PG_DATABASE_URL` for the PostgreSQL and TimescaleDB backend, `SQL_POOL_SIZE` for connection management, `DB_MIGRATIONS_DIR` for schema migration discovery, and `KAFKA_URL` for the Kafka bridge that persists live events. The shared configuration section centralizes these variables and their defaults, see **the environment configuration section**.

4.5.4 Components

Key components are:

- **Storage.** `TimescaleStorage` implements the `Storage` interface and encapsulates SQL read/write logic for sensors, readings, aggregates, and alerts.
- **Migrations.** `MigrationRunner` applies SQL migrations (tracked in `schema_migrations`) so the service can start against a known schema.
- **Kafka bridge.** `setup_bridge(...)` connects to Kafka, subscribes to all processed sensor topics and the alerts topic, and forwards each payload into storage.
- **REST API.** `create_database_blueprint(...)` exposes the storage operations via HTTP endpoints.

4.5.5 REST API surface

The database service exposes:

- `POST /bind_sensor` to register a `SensorDescriptor` and return a `sensor_id`.
- `GET /sensors` to list registered sensors.
- `GET /actuators` to list actuators.
- `GET /readings` to query either raw readings (`window=raw`) or aggregates (`window=1h`) based on a `ReadingsQuery`.
- `POST /alerts/definitions` to upsert an `AlertDefinition`.
- `GET /alerts/history` to query alert history.
- `GET /alerts/active` to retrieve currently active alerts.

4.5.6 Run command

Run the service via: `make run-database` which executes `dt/data/database/app.py` and starts Flask on port `5001`.

4.5.7 Failure modes and troubleshooting

If migrations fail, do not start the service against a partially initialized schema. Confirm `PG_DATABASE_URL` points to the expected database and inspect `dt/data/database/migrations/` for the canonical schema.

If the Kafka bridge cannot connect, `setup_bridge(...)` raises a connection error. In that case the REST API may still be reachable, but the database will not receive live updates from Kafka until the bridge is running.

If you see missing readings in the dashboard, validate the full pipeline: collector publishes → preprocessing publishes to `dt.sensors.processed.*` → database bridge ingests and persists → dashboard queries `/readings`.

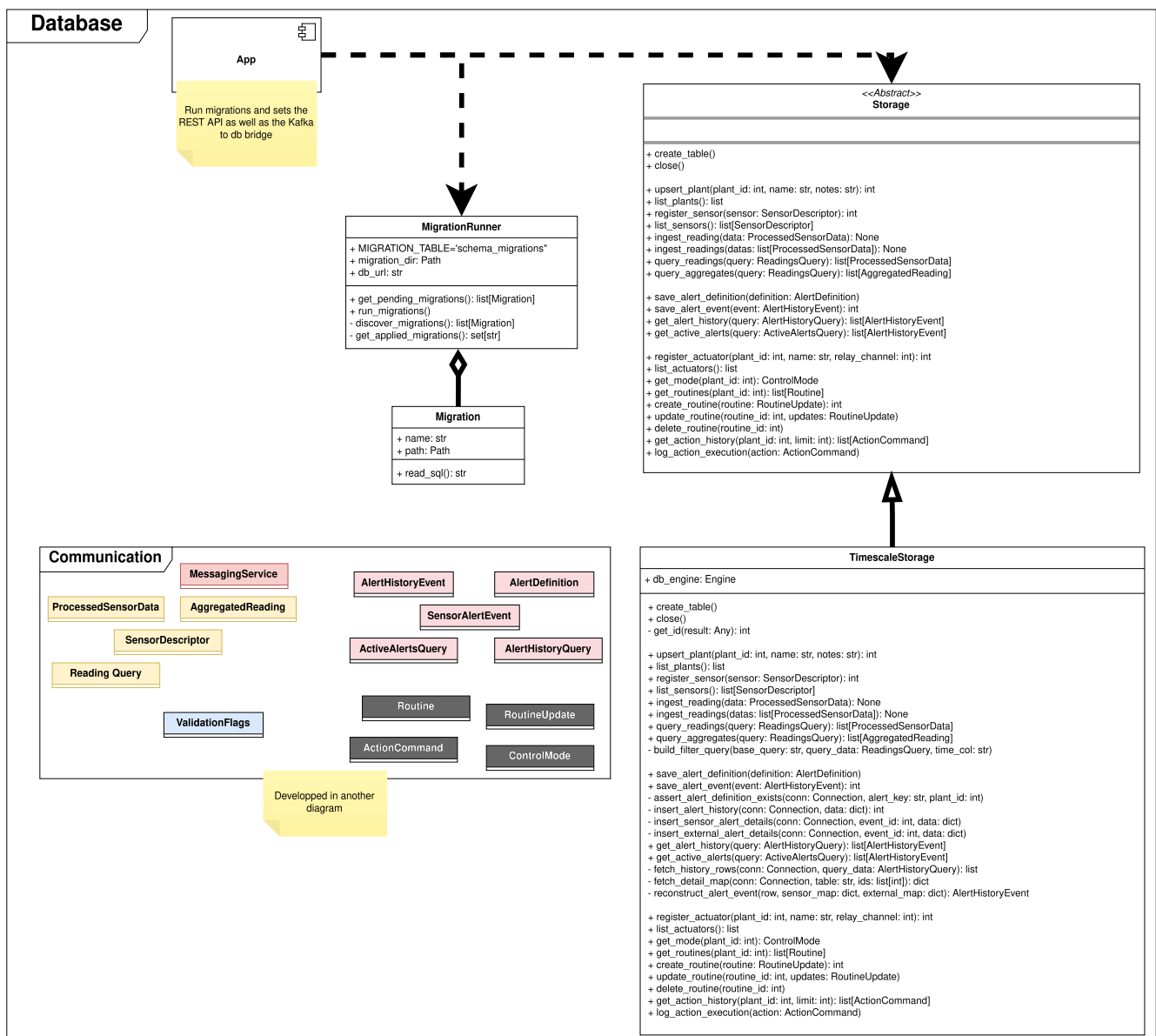


Figure 12 : Class diagram of the `dt/data/database` package, showing the main components: storage layer, migration runner

4.6 dt/webapp

The `dt/webapp` package is the operator-facing entry point of the platform. It provides a single HTTP origin for the dashboard UI and exposes a small REST API that the frontend can call to fetch

history and to issue control operations. In addition to request/response interactions, the webapp streams live telemetry to connected browsers using Socket.IO, so charts and alert panels update as soon as new Kafka messages arrive.

Operationally, the webapp sits “on top” of the rest of the system: it does not own any source of truth. Instead, it relies on the database service for historical queries and on the controller service for control state (mode, routines, actuator triggering). Its job is to translate those backend interfaces into browser-friendly payloads and to maintain a responsive, real-time UI.

4.6.1 Purpose

The webapp is responsible for

1. serving the dashboard UI,
2. exposing an API surface tailored to the frontend, and
3. bridging the event-driven backend (Kafka topics) to a push-based, client-friendly stream (Socket.IO events).

The REST API acts mostly as a proxy. One important piece of “glue” is timestamp normalization: browser requests and payloads use millisecond timestamps, while the database service returns seconds-based timestamps. `dt/webapp/api.py` centralises those conversions so the frontend does not need to care about backend timestamp conventions.

4.6.2 Inputs and outputs

The webapp has two main inputs:

- Kafka topics for live updates: sensor topics under `dt.sensors.processed.*` and alert events on `dt.alerts`.
- HTTP dependencies for query/control: the database service at `FLASK_DB_URL` and the controller service at `FLASK_CONTROLLER_URL`.

Its main outputs are HTTP responses and Socket.IO events:

- The REST API lives under `/api/`. It exposes historical queries like `GET /api/readings` (raw or 1h aggregates) and alert queries like `GET /api/alerts/active` and `GET /api/alerts/history`, plus metadata endpoints (`GET /api/sensors`, `GET /api/actuators`). Control-oriented endpoints proxy the controller service (`/api/control/mode`, `/api/routines`, `/api/actions/dispatch`, `/api/actions/history`).
- The Socket.IO stream uses event names that mirror the backend topics. For every processed sensor topic (for example `dt.sensors.processed.temperature`), the consumer emits a Socket.IO event with the same string name and a browser-shaped `ProcessedSensorData` payload. Alert events are emitted as `alerts_update` and `alerts_remove`.

To reduce “cold start” UX issues, `dt/webapp/consumer.py` maintains an in-memory cache of the last payload per processed topic and the set of active alerts. When a client connects, `dt/webapp/app.py` replays those cached payloads, so the UI renders a recent state without waiting for the next Kafka message.

4.6.3 Configuration

The webapp mainly needs to know where the rest of the platform lives. It uses `KAFKA_URL` for the live Socket.IO bridge, `FLASK_DB_URL` for historical data and alert state, and `FLASK_CONTROLLER_URL` for control operations. It also expects the built frontend to be present under `dt/webapp/static/ui`; if that directory does not contain `index.html`, the service returns a 503 for `/ui/`. The variable defaults are listed in **the environment configuration section**.

4.6.4 Components

`dt/webapp` is composed of a few small, focused modules:

- `dt/webapp/app.py` wires the Flask app, initialises Socket.IO, registers blueprints, and optionally starts the Kafka consumer bridge.
- `dt/webapp/api.py` implements the `/api/` blueprint (database queries and controller proxy endpoints) and owns the browser timestamp conversions.
- `dt/webapp/consumer.py` implements the Kafka → Socket.IO bridge, including cache management and active-alert seeding on startup.
- `dt/webapp/ui.py` serves the built Svelte SPA under `/ui/` using a “fallback to `index.html`” strategy for client-side routing.
- `dt/webapp/frontend/` contains the Svelte/Vite project that is built into `dt/webapp/static/ui`.

4.6.5 Run command

Run the dashboard service via `make run-dashboard` (module `dt.webapp.app`). Build the Svelte UI via `make build-webapp` so `/ui/` can serve the compiled assets.

4.6.6 Failure modes and troubleshooting

If `/ui/` returns a 503, the UI build output is missing. Run `make build-webapp` and confirm `dt/webapp/static/ui/index.html` exists.

If the UI loads but no live updates arrive, check Kafka connectivity. The webapp intentionally does not crash when Kafka is down; instead it reports a disconnected state and continues serving HTTP so operators can still access history.

If historical charts are empty while live data is present, verify the pipeline collector → preprocessing → database bridge is running and that `FLASK_DB_URL` points to the correct database service instance.

4.7 dt/controller

The `dt/controller` package is the closed-loop control part of the system. It decides when to actuate (based on sensor thresholds or time schedules) and it enforces safety rules before executing commands on physical actuators. From an operational perspective, it is the module that turns “observability” into “intervention”: once routines are defined and enabled, it continuously evaluates them and produces concrete actuator actions.

The controller is built around a clear separation of concerns. The REST API handles external requests (from the webapp or other clients), while the runtime service consumes the processed sensor stream from Kafka to evaluate sensor-based routines and runs a scheduler loop for time-based routines. Actuator execution is delegated to an `ActuatorManager` that applies policy constraints and records every execution in the database.

4.7.1 Purpose

The controller provides three capabilities:

1. **Control mode management.** It stores and retrieves a per-plant control mode (notably whether AI auto-pilot is enabled) and uses that mode to decide whether routines should be allowed to actuate.
2. **Routine management.** It accepts “logic builder” routine graphs, validates them as directed acyclic graphs, compiles them into a flat list of executable rules, and persists those compiled rules in the database for runtime use.

3. **Action execution.** It executes manual, AI, or routine-triggered actions on actuators while enforcing safety policies (cooldowns, duration limits, and allowed commands), and it logs outcomes for auditability.

4.7.2 Inputs and outputs

The controller consumes the processed sensor stream from Kafka. On startup, `ControllerService.start()` connects to Kafka (`KAFKA_URL`) and subscribes to every processed topic produced by `Topics.list_sensor_topics()` (for example `dt.sensors.processed.soil_moisture`). Each incoming `ProcessedSensorData` payload is evaluated against compiled “sensor triggers”.

Time-based triggers (time/date/interval) are handled separately. A scheduler thread runs once per second and evaluates minute-aligned points between the last check and “now”, so the controller does not skip scheduled events even if the loop lags briefly or the service restarts.

Externally, the controller exposes a REST API under `/controller/` (Flask port 5004 by default) for mode, routine, and action operations. Additionally, it logs all action outcomes to the database service via `DatabaseApiClient` (including rejected, failed, and skipped actions).

4.7.3 Configuration

Key operational settings come from `dt/utils/config.py`:

- `FLASK_DB_URL` configures the database service used for mode/routine storage, actuator binding, and action execution logging.
- `KAFKA_URL` configures the Kafka broker used to consume processed sensor topics and publish action events.
- `TIMEZONE` controls how time-based routines are interpreted (for example, a “07:30” trigger fires at 07:30 in this timezone, not in UTC).
- `ACTUATOR_POLICIES_PATH` points to the YAML file that defines actuator policy constraints. If the file is missing or invalid, the controller falls back to conservative defaults. The full policy model is described in **the dedicated actuator policy section**

4.7.4 Components

The controller runtime is structured around a small set of components:

- `dt/controller/app.py` wires the Flask app, builds the actuator set, and starts the `ControllerService` alongside the HTTP API.
- `dt/controller/api.py` implements the REST endpoints for mode, routine CRUD, and action dispatch/history.
- `dt/controller/service.py` is the orchestrator: it loads compiled routines from the database, consumes sensor events, runs the scheduler loop for time triggers, and dispatches actions for execution.
- `dt/controller/routine_compiler.py` validates and compiles routine graphs using `networkx`. It enforces DAG-ness and requires that every action node is reachable from at least one trigger.
- `dt/controller/routine_evaluator.py` contains the pure “decision logic” for mapping a triggered rule to one or more `ActionCommand` instances.
- `dt/controller/policies.py` loads and resolves `ActuatorConfig` constraints from YAML.
- `dt/controller/actuator_manager.py` enforces policies, executes actions on concrete actuator drivers (`dt/controller/kinds/*`), and records outcomes.

4.7.5 Run command

Run the controller service via `make run-controller` (module `dt.controller.app`).

4.7.6 Failure modes and troubleshooting

If the controller cannot connect to Kafka on startup, `ControllerService.start()` raises and no routine evaluation will occur. Verify `KAFKA_URL` and that the processed topics exist.

If routines never fire, check (1) that they are enabled and compiled in the database, (2) that the plant is not in AI auto-pilot mode (routines are marked as skipped when auto-pilot is enabled), and (3) that `TIMEZONE` matches the operator's expectations for time-based triggers.

If actions are consistently rejected, inspect the resolved actuator policy (cooldown, max duration, allowed commands) and verify that actuator binding succeeded in the database. When the policy file is missing, defaults are used, which can be more restrictive than intended.

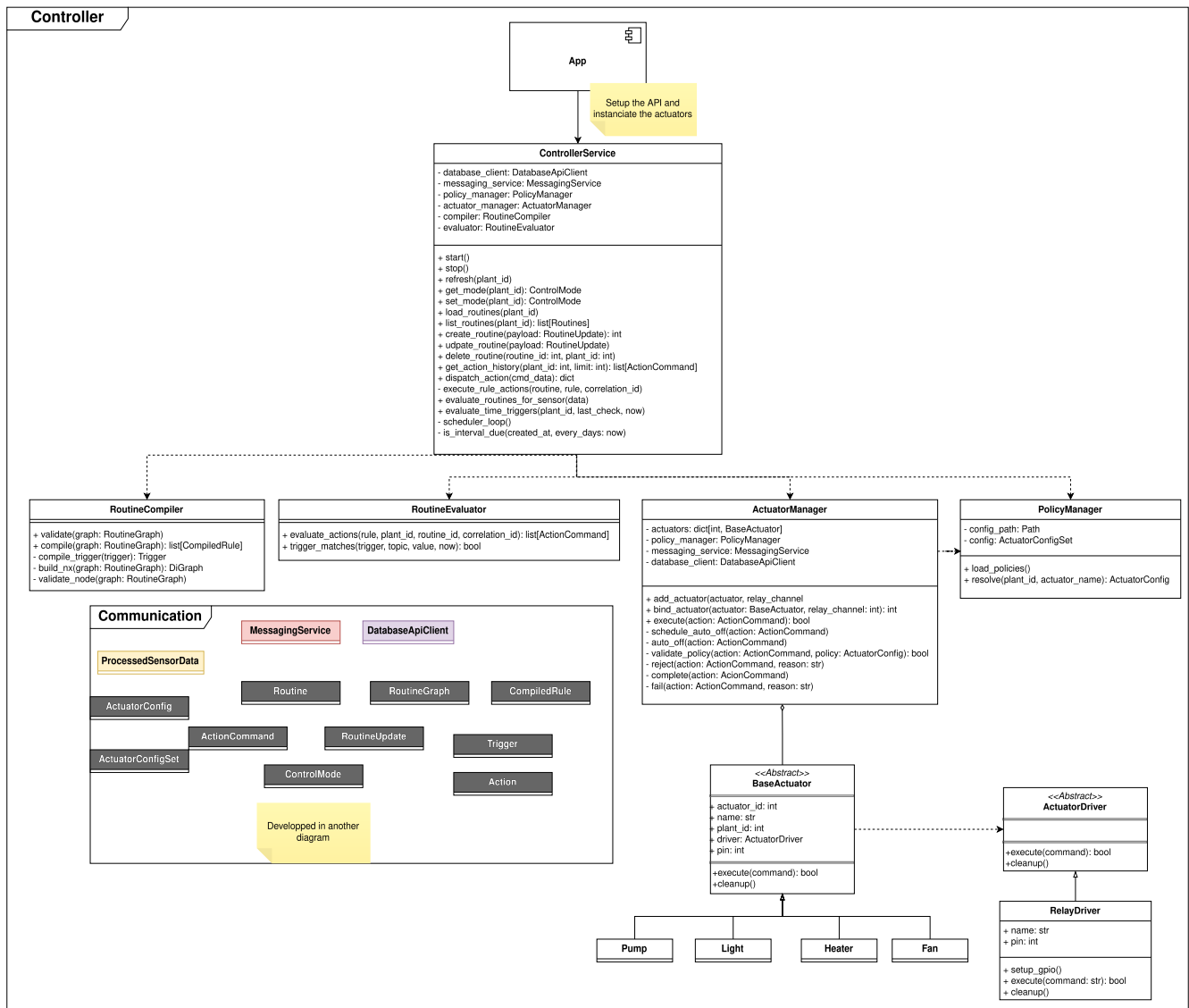


Figure 13 : Class diagram of the `dt/controller` package, showing the main components: app, API, service, routine compiler/evaluator, policies, and actuator manager.

5 Configuration

The configuration model follows a simple rule: infrastructure settings live in environment variables, while service behaviour that changes often is described in YAML files under `dt/utils/`. This split keeps deployment concerns and domain tuning separate. You edit `.env`

when services must talk to different hosts or use different runtime limits, and you edit YAML when you want the system to interpret data or actuators differently.

In practice, configuration should be applied in three passes. First, define the shared infrastructure in `.env` so every process points to the same Kafka broker and the same HTTP and database endpoints. Second, tune the service-level behaviour in the YAML files used by preprocessing, alerting, and control. Third, restart the affected process, because these files are loaded at startup rather than watched dynamically.

5.1 Environment variables (`.env`)

The common runtime settings are defined in `dt/utils/config.py` through the `Config` enum. The module calls `load_dotenv()` on import, so a `.env` file at the repository root is loaded automatically when the services are started through `poetry run` or the Make targets.

The repository includes `.env.example` as a starting point. For a single-machine setup, most defaults are usable as-is. As soon as the services are distributed across several hosts, these values should be set explicitly so every module still targets the same broker, the same database, and the same companion services.

```
1 # Kafka
2 KAFKA_URL=localhost:9092
3
4 # PostgreSQL + TimescaleDB
5 PG_DATABASE_URL=postgresql+psycopg://dt:dt@localhost:5432/dt
6
7 # Service endpoints
8 FLASK_DB_URL=http://localhost:5001/
9 FLASK_DASHBOARD_URL=http://localhost:5000/
10 FLASK_AI_URL=http://localhost:5002/
11 FLASK_CONTROLLER_URL=http://localhost:5004/
```

 DotENV

Variable	Default	Purpose
KAFKA_URL	localhost:9092	Kafka bootstrap server used by all producers and consumers.
PG_DATABASE_URL	postgresql+psycopg://dt:dt@localhost:5432/dt	Database URL used by the storage service and by any direct storage client.
SQL_POOL_SIZE	5	Connection pool size used by the TimescaleDB storage adapter.
DB_MIGRATIONS_DIR	dt/data/database/migrations	Directory containing SQL migrations applied by the database service and by scripts/ <code>run_sql_migration.py</code> .
FLASK_DASHBOARD_URL	http://localhost:5000/	Base URL of the dashboard service.

FLASK_DB_URL	http://localhost:5001/	Base URL of the database service REST API.
FLASK_CONTROLLER_URL	http://localhost:5004/	Base URL of the controller service REST API.
PREPROCESSING_CONFIG_PATH	dt/utils/preprocessing_config.yml	Path to the preprocessing configuration YAML.
PREPROCESSING_CHECKPOINT_DIR	./spark-checkpoints/preprocessing	Writable checkpoint directory for Spark offsets and state.
MAX_STATE_HISTORY_LENGTH	64	Maximum number of history entries retained per sensor in preprocessing state.
SPARK_APP_NAME	dt-preprocessing-app	Spark application name.
SPARK_MASTER	local[2]	Spark master URL or local execution mode.
SPARK_SQL_SHUFFLE_PARTITIONS	2	Number of partitions used by Spark shuffle stages.
SPARK_DEFAULT_PARALLELISM	2	Default Spark parallelism when no explicit partition count is provided.
SPARK_AQE_ENABLED	false	Enable or disable Spark Adaptive Query Execution.
SPARK_MAX_OFFSETS_PER_TRIGGER	500	Maximum number of Kafka offsets consumed per Structured Streaming trigger.
SPARK_TRIGGER_INTERVAL	5 seconds	Processing-time interval between Spark micro-batches.
SPARK_WATERMARK_INTERVAL	10 minutes	Allowed lateness before events are considered too old for stateful work.
SPARK_STATE_TIMEOUT_SECONDS	600	Timeout for inactive per-key Spark state.
SPARK_LOG_LEVEL	WARN	Spark log level.
STARTING_OFFSETS	latest	Kafka starting offsets for the preprocessing job.
ALERT_RULES_PATH	dt/utils/alert_rules.yml	Path to the alert rules YAML loaded by the alert engine.
ACTUATOR_POLICIES_PATH	dt/utils/actuator_policies.yml	Path to the actuator policies YAML loaded by the controller.

TIMEZONE	Europe/Brussels	Timezone used when evaluating time-based routines.
----------	-----------------	--

5.2 Configuration responsibilities by module

Once the shared environment is in place, each module only needs a small and well-defined subset of that configuration surface.

- `dt/communication` does not own runtime configuration. It carries the shared contracts and the HTTP/Kafka helper clients that consume the shared environment variables of the services using them.
- `dt/collector` mainly depends on `KAFKA_URL` and `FLASK_DB_URL`. The sensor inventory itself is currently wired in code in `dt/collector/main.py`, so changing hardware pin assignments or read intervals is still a code change.
- `dt/data/preprocess` combines shared infrastructure settings with the detailed YAML described in **the preprocessing configuration section**.
- `dt/alerts` combines `KAFKA_URL`, `FLASK_DB_URL`, and the YAML described in **the alert rules section**.
- `dt/data/database` uses `PG_DATABASE_URL`, `SQL_POOL_SIZE`, `DB_MIGRATIONS_DIR`, and `KAFKA_URL`. It does not rely on a separate YAML file.
- `dt/webapp` uses `KAFKA_URL`, `FLASK_DB_URL`, and `FLASK_CONTROLLER_URL`. Its browser assets are served from `dt/webapp/static/ui`, which must contain a built frontend.
- `dt/controller` uses `KAFKA_URL`, `FLASK_DB_URL`, `TIMEZONE`, and the YAML described in **the actuator policies section**.

This division matters operationally because it tells you where to look first. If multiple services fail to communicate, start with `.env`. If only one service behaves incorrectly while the infrastructure is healthy, inspect the module-specific YAML or code-level wiring for that service.

5.3 Preprocessing config (YAML)

The structure of the file mirrors the way the pipeline reasons about a sensor. It starts with global conventions, then defines reusable templates, and only then binds concrete sensors to those templates:

- Under `system`, the file defines defaults shared across sensors, most notably the weights used to compute the data-quality score. These weights should sum to `1.0`, because they express how much each validation dimension contributes to the final `dq_score`.
- Under `templates`, the file describes reusable sensor profiles keyed by a logical sensor type and metric such as `dht22.temperature` or `stemma.soil_moisture`. A template can define units and any combination of stage settings for validation, imputation, smoothing, calibration, and normalization.
- Under `sensors`, the file binds actual deployed sensors to a template and adds only the overrides that are specific to that device (sensor may have drifted over time). This keeps the common rules in one place while still allowing sensor-by-sensor calibration and threshold tuning.

The binding step deserves special attention because the incoming events carry a numeric `sensor_id`, while the YAML keys are strings. On startup, the preprocessing service queries the database service, reads the registered sensor names, and builds a registry from `sensor_id` to YAML key. In the common case, that key is the sensor name stored in the database (for example

`sensors.basil.dht22.001.temperature`). If no exact match exists, the job can fall back to a generic key derived from the name (for example `dht22.temperature`)

If a sensor cannot be resolved to a configuration key, the preprocessing job raises `KeyError` (“No sensor registry entry”). In that case, either bind the sensor in the database with a name that matches your YAML, or add a matching entry under `sensors:` . The following example shows the intended pattern: a reusable template plus a short override for one physical sensor.

```
1  system:
2    windows:
3      small_sec: 60
4      medium_sec: 300
5      big_sec: 1000
6    weights:
7      range_ok: 0.4
8      roc_ok: 0.4
9      stuck_ok: 0.2
10
11  templates:
12    dht22.temperature:
13      units: "°C"
14      validation:
15        range: { min: -40, max: 80 }
16        roc: { max_per_minute: 1.0 }
17        # roc:
18        #   profiles:
19        #     indoor: { max_per_minute: 2.0 }
20        #     outdoor: { max_per_minute: 8.0 }
21        #   active_profile: indoor
22        # stuck: { max_flat_seconds: 300 }
23      imputation:
24        strategy: window_average
25        window_seconds: 360
26        min_samples: 2
27      smoothing:
28        strategy: pass_through
29      calibration:
30        strategy: affine
31        scale: 1.0
32        offset: 0.0
33      normalization:
34        strategy: min_max
35        input_min: -40.0
36        input_max: 80.0
37        output_min: 0.0
38        output_max: 1.0
39        clip: true
```

YAML


```

40
41 sensors:
42   dht22.temperature: { template: dht22.temperature }
43   sensors.basil.dht22.001.temperature:
44     template: dht22.temperature
45     calibration:
46       strategy: affine
47       offset: -0.2

```

The supported configuration blocks stay intentionally small so the runtime behaviour remains understandable: **Validation.** `validation.range` enforces inclusive `min` / `max` bounds. `validation.roc` enforces a maximum absolute delta per minute, either as a single `max_per_minute` limit or as named `profiles` with an `active_profile`. `validation.stuck` detects flatlines using `max_flat_seconds` and the rolling history maintained by the job. Validation results are surfaced through `flags` (violations) and the derived `dq_score` computed from `system.weights`.

Imputation. Imputation runs only when a reading is invalid. The strategy is selected by `imputation.strategy`:

- `window_average` averages the recent valid history inside `window_seconds` and requires `min_samples`
- `forward_fill_with_decay` carries the last valid value forward, decaying toward an optional `baseline` when the gap grows, with `max_gap_seconds` and `decay_seconds` as guardrails
- `linear_extrapolation` projects the recent trend using `window_seconds` and an optional `max_gap_seconds`.

Smoothing. Smoothing is applied after imputation so that both real and imputed points can be stabilised consistently. The default is `pass_through` (no additional filtering). When configured as `ewma`, the smoothing stage applies an exponentially weighted moving average controlled by `alpha` ($0 < \alpha \leq 1$).

Calibration. Calibration transforms the raw sensor output into a meaningful scale before validation:

- `identity` leaves values unchanged
- `affine` applies `scale` and `offset`
- `piecewise_lookup` maps input ranges to constant outputs using a list of segments, which is useful for sensor-specific lookup tables.

Normalization. Normalization is intended for downstream analytics that expect bounded inputs:

- `identity` leaves values unchanged
- `min_max` maps from `[input_min, input_max]` to `[output_min, output_max]` and can `clip` values outside the input range.

The pipeline keeps both the final `value` and the separate `normalized_value` in the emitted payload for traceability.

Configuration changes require restarting the preprocessing job to take effect.

5.4 Alert rules (YAML)

Like the preprocessing configuration, the rules file (default: `dt/utils/alert_rules.yml`, overridden via `ALERT_RULES_PATH`) is loaded at startup. A rule change is therefore not live until the alert engine is restarted.

The file is a single mapping with an `alert_rules:` list. Each entry answers four questions in order: which data stream is evaluated, which condition is applied, how severe the resulting alert is, and how much noise the engine tolerates before it emits or re-emits that alert.

The alert engine evaluates rules per incoming processed event. When a condition matches, the engine creates a candidate alert key as `<rule_id>:<source>`. The registry then applies `persistence_count` (require N consecutive matches before emitting) and `cooldown_seconds` (suppress re-firing within a time window). This is why a rule may be “ignored” during short spikes even when the threshold is technically crossed.

```
1 alert_rules:
2   - rule_id: dq_score_low
3     name: Low Data Quality
4     description: "DQ score {dq_score} below {threshold} (sensor {sensor_id})"
5     severity: info
6     evaluation_stage: processed
7     source: "*"
8     condition:
9       type: dq_score
10      threshold: 0.7
11    persistence_count: 1
12    cooldown_seconds: 120
```

YAML

The `source` field scopes the rule to one processed signal family such as `temperature`, `humidity`, `soil_moisture`, or `light_intensity`, or to all of them through `*`. The `condition` block then defines the actual trigger:

- `threshold` compares the numeric `value` against `threshold` using an `operator` in `{>, <, >=, <=, ==, !=}`.
- `range` triggers when the numeric `value` is outside `[min_value, max_value]` (use `null` to express one-sided bounds).
- `dq_score` triggers when the DQ score is below `threshold`.
- `validation_flag` triggers when a specific `flags` entry equals `expected`, `flag` must be one of `range_violation`, `rate_of_change_violation`, `stuck_violation`, or `valid_data_point`.

Rule descriptions are Python format strings. Placeholders are filled using both the processed payload fields (`{value}`, `{sensor_id}`, `{timestamp}`, `{dq_score}`, ...) and the condition parameters (`{threshold}`, `{min_value}`, ...).

Operationally, the alert engine also depends on `KAFKA_URL` and `FLASK_DB_URL` from **the shared environment**, because rule evaluation only makes sense if the service can both consume processed sensor events and hydrate its state against the database service.

5.5 Actuator policies (YAML)

Actuator policies are loaded by the controller from `dt/utils/actuator_policies.yml`, or from another path when `ACTUATOR_POLICIES_PATH` is set. This file does not define the control logic itself. Instead, it defines the safety policies against which the controller is allowed to execute commands.

The schema is intentionally narrow. It starts with `defaults`, which apply to every actuator unless overridden, and then adds an `actuators` mapping keyed by actuator name for the exceptions.

The supported fields are the ones the runtime actually enforces:

- `max_duration_seconds` caps the duration of a single accepted ON action.
- `min_cooldown_seconds` enforces a minimum delay between accepted actions.
- `allow_overlap` determines whether two non-OFF actions may overlap.
- `allowed_commands` restricts the accepted command strings.

Policy resolution is layered so the service always has a usable answer. The controller starts from built-in conservative defaults, overlays the values from `defaults`, and then applies any actuator-specific override from `actuators.<name>`. If the policy file is missing or invalid, the controller falls back to those conservative built-in defaults.

The enforcement rules in `ActuatorManager` are straightforward. OFF commands always bypass policy checks. Non-OFF commands must be in the allowed command list when such a list is configured. Duration and cooldown are checked before execution, and overlap is rejected unless explicitly permitted.

```
1 defaults:
2   max_duration_seconds: 30
3   min_cooldown_seconds: 10
4   allow_overlap: false
5   allowed_commands: ["ON", "OFF"]
6
7 actuators:
8   pump:
9     max_duration_seconds: 5
10    min_cooldown_seconds: 120
11    allow_overlap: false
12  light:
13    max_duration_seconds: 43200
14    min_cooldown_seconds: 0
15    allow_overlap: false
16  fan:
17    max_duration_seconds: 600
18    min_cooldown_seconds: 20
19  heater:
20    max_duration_seconds: 300
21    min_cooldown_seconds: 30
```

YAML

The controller also needs `KAFKA_URL`, `FLASK_DB_URL`, and `TIMEZONE` from **the shared environment**.

6 Storage

This section explains how storage works operationally: where the system persists data, how the dashboard and services query it, how schema changes are deployed, and what to expect from retention and aggregation policies.

The storage backend is a single PostgreSQL database with the TimescaleDB extension enabled. In practice, this database serves two purposes at once: it is the time-series store for sensor telemetry and also the relational store for domain state (plants, sensors, actuators, routines, controller modes, and alert lifecycle history). Keeping both in one database avoids cross-store joins and operational split-brain: there is one schema, one backup surface, and one place to look when debugging “what happened”.

6.1 What is stored where

At a high level, the rule of thumb is: anything that is “event-like” and needs fast time-range queries goes into TimescaleDB, while anything that represents a stable piece of domain state or a business entity lives in regular relational tables.

Time-series telemetry is stored in the `sensor_readings` hypertable (created in `dt/data/database/migrations/001_init.sql`). Each row represents a single processed measurement received by the database service from Kafka. In addition to the main value fields (`timestamp`, `value`, `unit`), rows carry operational/audit context:

- `plant_id`, `sensor_id`, and `topic` so readings can be filtered without extra lookups
- `correlation_id` to trace an end-to-end flow across from the collector to the alert-engine and controller.
- data quality fields (`dq_score`, `flags`, `imputed`)
- optional fields (`raw_value`, `calibrated_value`, `normalized_value`) and the associated profile IDs, which are useful when investigating why a reading looks the way it does.

For longer lookback windows, the database maintains a continuous aggregate, `sensor_readings_1h`. This materialized view rolls up the hypertable into one-hour buckets (avg/min/max, sample count, average DQ score, and an imputed-sample count). Operationally, the UI and analytical consumers should prefer this view when they do not need raw resolution.

Relational tables store the domain model and lifecycle history:

- `plants`, `sensors`, and `actuators` store inventory/configuration that changes rarely compared to telemetry.
- `controller_modes` and `routines` store controller state and user-defined automation logic.
- `action_executions` is an audit log for actuator commands (rejected/running/completed/failed/skipped, plus timestamps and correlation ID).
- Alerts are split into an invariant definition table and an append-only history table: `alert_definitions` is keyed by (`alert_key`, `plant_id`), `alert_history` stores each lifecycle event (triggered/acknowledged/cleared) with timestamps, severity, message, and correlation ID. Event-specific details are stored alongside history rows in `alert_sensors` (a snapshot of the sensor reading context that triggered the alert) and `alert_external` (JSONB metadata for external alerts).

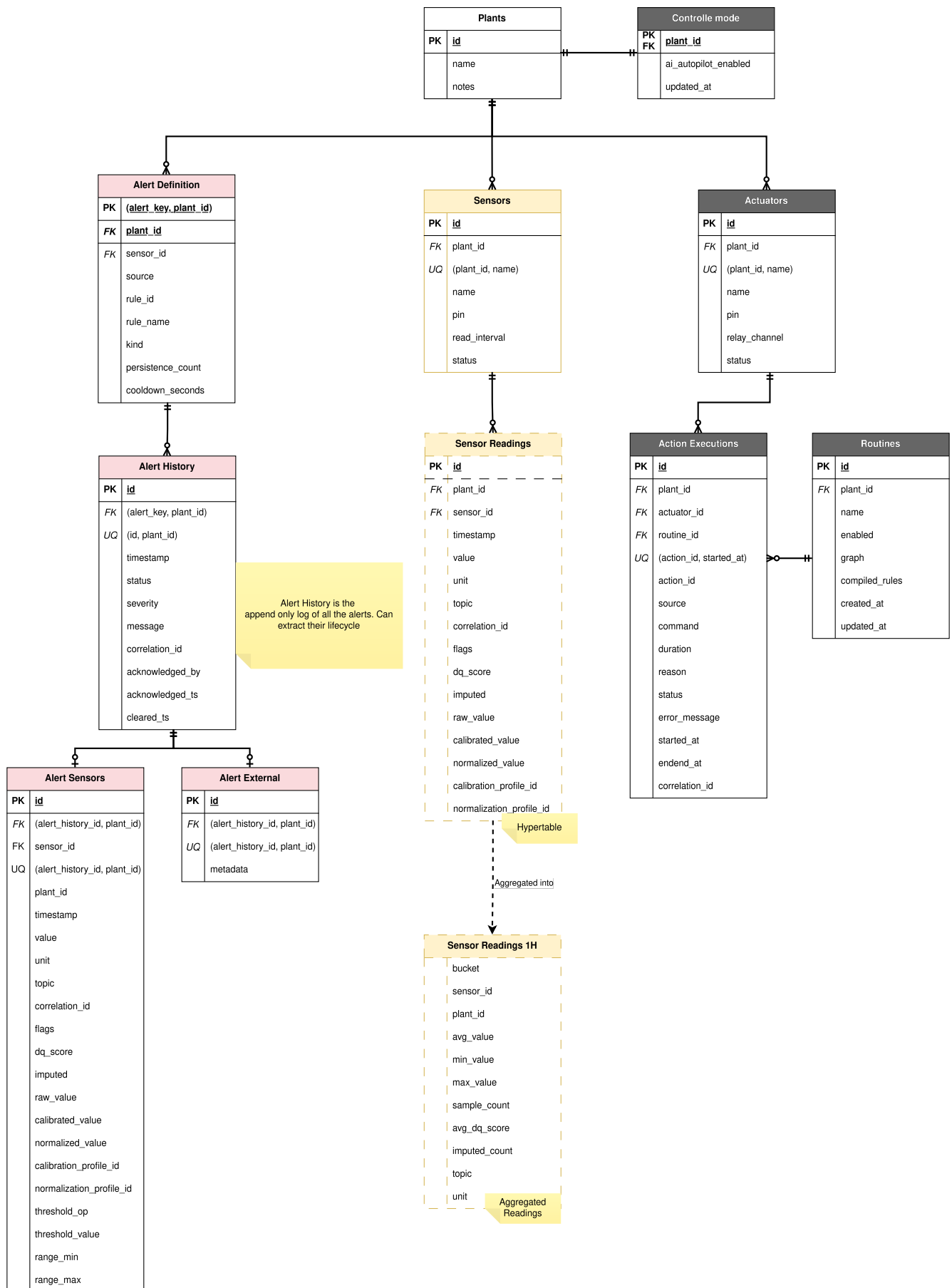


Figure 14 : Database schema diagram, showing the main tables for sensors, readings, and alerts, and their relationships.

6.2 Query patterns

Most operational queries come from the dashboard. They are served by the database service (`dt/data/database/app.py`) via HTTP endpoints that delegate directly to `TimescaleStorage` (`dt/data/database/timescale_storage.py`).

The core endpoint is `GET /readings` . It supports two modes via the `window` parameter:

- `window=raw` queries the hypertable (`sensor_readings`).
- `window=1h` queries the continuous aggregate (`sensor_readings_1h`).

In both cases the same filter parameters apply: `plant_id` , `sensor_id` , `topic` , and a time range (`since` , `until`). The `since` and `until` values are Unix timestamps in seconds (`float`), and are converted server-side using `to_timestamp(...)` . For predictable performance, always provide a bounded time window; unbounded queries can return very large result sets.

A typical dashboard pattern is to use raw readings for live ranges (minutes to a few hours), then switch to the 1-hour aggregates for longer history. The database service orders results ascending by time (`timestamp` for raw rows, `bucket` for aggregates), which matches the way charting components usually consume data.

Alerts are queried through two complementary endpoints:

- `GET /alerts/history` returns an append-only log of past alert events (most recent first, optional `plant_id` , with a `limit`).
- `GET /alerts/active` returns the currently active alerts by selecting the latest event per (`alert_key` , `plant_id`) and filtering out alerts whose latest status is `cleared` .

The controller module introduces a second family of operational queries. The dashboard reaches them through the webapp proxy, but the underlying state is persisted in the database service and read back by the controller service. In practice, three GET endpoints matter:

- `GET /controller/mode` returns the current control mode for a plant, filtered by `plant_id` .
- `GET /controller/routines` returns the routines configured for a plant, also filtered by `plant_id` .
- `GET /controller/actions/history` returns recent action execution records for a plant, filtered by `plant_id` and optionally bounded by `limit` (default 50).

These queries complement telemetry and alerts rather than replacing them. Readings and alert history explain what the plant experienced, while the controller queries explain how automation was configured and which actions were attempted or executed in response. Action history is ordered most recent first (`started_at DESC`), which makes it suitable for operator timelines and recent activity panels.

6.3 Schema management

Schema changes are managed as SQL migrations under `dt/data/database/migrations/` . Migrations are applied in alphanumeric order and tracked in a dedicated `schema_migrations` table, which allows the runner to be safely re-executed without reapplying the same file.

Operationally there are two ways migrations run:

First, the database service runs migrations on startup (see `run_startup_migrations(...)` in `dt/data/database/app.py`). This is the default path: if migrations fail, the service should not continue against a partially initialized schema.

Second, the same migration logic is available as a manual CLI entry point via `python scripts/run_sql_migration.py`. This is useful when you want to initialize a database before starting the service, or when debugging a failed migration in isolation.

Migrations are expected to be idempotent and safe to rerun. Concretely, that means using patterns like `CREATE TABLE IF NOT EXISTS`, and Timescale policy statements that use `if_not_exists => TRUE` where supported. When changing an existing object (table shape, policy parameters, constraints), create a new numbered migration file rather than editing an already-applied migration, so environments converge deterministically.

There is no automatic rollback mechanism. If a migration introduces a bad change, the operational recovery path is to restore from backup or to ship a follow-up migration that corrects the schema.

6.4 Data retention

Raw telemetry retention is defined as part of schema initialization. In `001_init.sql`, the hypertable `sensor_readings` is configured with a retention policy that keeps 30 days of raw data by default. The intent is to keep the raw resolution window large enough for debugging and recent analysis, while relying on aggregates for longer-term trends.

Downsampling is handled through continuous aggregates rather than ad-hoc batch jobs. The `sensor_readings_1h` view provides the canonical “long lookback” representation, and its refresh policy is defined in the same migration (refreshing a trailing window on a fixed schedule). In other words: retention shrinks raw data over time, and the aggregate view is what preserves the historical shape of the signal.

If you need to add or adjust retention and downsampling for a deployment, do it in SQL and ship it as a numbered migration. The TimescaleDB APIs are “policy-based”: you declare what should happen (retain raw data for N days, refresh aggregates on a schedule), and TimescaleDB creates background jobs to enforce it.

To add a retention policy for a hypertable, use `add_retention_policy`. The example below keeps 90 days of raw readings:

```
1 SELECT add_retention_policy(  
2     'sensor_readings',  
3     INTERVAL '90 days',  
4     if_not_exists => TRUE  
5 );
```

SQL

To add a downsampling policy, the common pattern is: create a continuous aggregate at the desired resolution (for example 5 minutes), then add a continuous aggregate policy to refresh it periodically. This is the same mechanism used by `sensor_readings_1h` in `001_init.sql`.

```
1 CREATE MATERIALIZED VIEW IF NOT EXISTS sensor_readings_5m  
2 WITH (timescaledb.continuous) AS  
3 SELECT  
4     time_bucket('5 minutes', timestamp) AS bucket,  
5     sensor_id,  
6     plant_id,  
7     topic,  
8     unit,
```

SQL

```

9  AVG(value) AS avg_value,
10 MIN(value) AS min_value,
11 MAX(value) AS max_value,
12 COUNT(*) AS sample_count,
13 AVG(dq_score) AS avg_dq_score,
14 SUM(CASE WHEN imputed THEN 1 ELSE 0 END) AS imputed_count
15 FROM sensor_readings
16 GROUP BY bucket, sensor_id, plant_id, topic, unit
17 WITH NO DATA;
18
19 SELECT add_continuous_aggregate_policy(
20     'sensor_readings_5m',
21     start_offset => INTERVAL '2 hours',
22     end_offset => INTERVAL '5 minutes',
23     schedule_interval => INTERVAL '5 minutes',
24     if_not_exists => TRUE
25 );

```

In addition to retaining raw data, you may also want to retain aggregates (or keep them indefinitely). Continuous aggregates store their materialized data in an internal hypertable. If you decide to expire aggregate data, first look up the materialization hypertable name and then apply a retention policy to that hypertable.

```

1  SELECT
2     view_name,
3     materialization_hypertable_name
4  FROM timescaledb_information.continuous_aggregates
5  WHERE view_name IN ('sensor_readings_1h', 'sensor_readings_5m');

```

SQL

TimescaleDB storage optimization is enabled at the hypertable level via Hypercore settings (column store enabled, chunk interval, and segment/order keys). The column store allow for better compression of time-series data without losing any detail.

When operating the system, the safest way to change retention or aggregation behavior is to add a new migration that updates policies. Avoid one-off manual changes in production: they tend to drift from what the repository declares, and the next migration run can surprise you.

7 Guidelines

This chapter is written for contributors who need to change the system, not just run it. The codebase is intentionally modular, but that modularity only pays off when extensions follow the existing contracts: topic naming, dataclass payload shapes, configuration keys, and the “one place to define it” principle.

7.1 General coding standards

The most important principle in this codebase is that **contracts are shared** and **side effects are localized**. Dataclasses and topic names live under `dt/communication/` and are meant to be imported everywhere. By contrast, anything that opens a network connection, touches hardware, or performs I/O belongs in the relevant service package (`dt/collector`, `dt/data/preprocess`, `dt/alerts`, `dt/data/database`, `dt/webapp`) so that tests and imports stay predictable.

When you introduce a change, always think in terms of the full path a payload takes: “what is emitted?”, “who consumes it?”, and “where is it stored?”. In practice this means:

1. If you touch a payload shape (e.g., `ProcessedSensorData`), you also need to check adapters (`dt/communication/adapters/*`), storage persistence, and UI shaping.
2. If you touch a topic name, you need to update both producers and consumers, and you must keep the topic derivation rules consistent (`Topics.<name>.raw` / `Topics.<name>.processed`).
3. If you change configuration schemas (preprocessing YAML or alert rules YAML), you must update the typed loaders that validate those files and the tests that assert the schema behaviour.

Naming is not cosmetic in this system: it drives routing and configuration lookup. The key conventions to follow are:

- Kafka topics are centralized in `dt/communication/topics.py` and sensor topics live under the `dt.sensors.*` prefix. Raw and processed variants are derived mechanically.
- Sensor configuration keys are strings stored in the database as `SensorDescriptor.name`. The preprocessing job resolves a numeric `sensor_id` back to a configuration key by calling the database service on startup. It prefers an exact match, and otherwise derives a generic key of the form `<sensor_type>.<metric>`.

In other words, these two keys can coexist:

```
1 sensors.basil.dht22.001.temperature # specific, per-device override
   (stored in DB as the sensor name)
2 dht22.temperature                  # generic default (used as fallback)
```

Plain Text

Finally, error handling must happen at the boundaries:

- hardware reads should be allowed to fail **without corrupting the pipeline** (return `None` to indicate “no data” rather than publishing sentinel values that look like real measurements)
- REST endpoints must validate request payloads and return clear 4xx errors for invalid input
- storage operations should fail loudly and deterministically, partial writes are worse than no writes because they produce misleading dashboards

The `correlation_id` is the trace handle across services. It is created when the collector emits a new reading and should be preserved end-to-end so you can link “a sensor event” to “a database row” to “an alert event” to “an action”.

7.2 Adding a sensor

Adding a sensor is a cross-cutting change because a new physical input becomes a new data stream, then a new preprocessing configuration target, and finally a new visualization.

The goal is that the new sensor behaves like the existing ones: it binds once, publishes raw events on a stable schedule, is processed by the standard pipeline, and becomes visible in the UI without “special cases” spread across the system.

7.2.1 Add the topic (the routing contract)

Start by adding a new `Topics` member in `dt/communication/topics.py`. Every consumer that works “for all sensors” discovers sensors via `Topics.list_sensor_topics()`, so if the topic is not declared here, downstream services will not subscribe to it.

```
1 from dt.communication.topics import Topics
2
```

Python

```

3 # Example: add a new sensor topic (edit dt/communication/topics.py)
4 # C02 = PREFIX_SENSOR + "co2"

```

The sensor will publish to `Topics.<your_topic>.raw`. The preprocessing job and webapp subscribe to raw/processed patterns derived from these topic names, and the alert engine subscribes to the processed equivalents.

7.2.2 Implement the collector sensor class (the hardware adapter)

Create a new sensor implementation under `dt/collector/kinds/` by subclassing `Sensor` (`dt/collector/kinds/base_sensor.py`). Your class is responsible for:

- defining a stable `unit` (what the value means)
- selecting the correct `topic` from `Topics` (what stream it belongs to)
- returning a `float` measurement from `read_sensor()`, or `None` when no measurement is available

```

1  from typing_extensions import override
2
3  from dt.collector.kinds.base_sensor import Sensor
4  from dt.communication.topics import Topics
5
6
7  class Co2Sensor(Sensor):
8      def __init__(self, name: str, read_interval: int, pin: int) -> None:
9          super().__init__(name, read_interval, pin)
10         self._unit = "ppm"
11         # self._sensor = ...
12
13     @property
14     @override
15     def unit(self) -> str:
16         return self._unit
17
18     @property
19     @override
20     def topic(self) -> Topics:
21         return Topics.C02
22
23     @override
24     def read_sensor(self) -> float | None:
25         value = None # read from hardware
26         return value

```

 Python

The collector will bind the sensor to the database (to obtain a numeric `sensor_id`) and will publish `RawSensorData` events containing `plant_id`, `sensor_id`, `timestamp`, `value`, `unit`, `topic`, and `correlation_id`.

7.2.3 Wire it into the collector endpoint (the runnable service)

Add an instance in `dt/collector/main.py` and register it with `SensorManager`. On startup, the manager calls the database service (`POST /bind_sensor`) with a `SensorDescriptor` derived from the sensor. The returned ID is then attached to each emitted reading.

This binding step is what allows preprocessing to resolve the numeric `sensor_id` back to a YAML configuration key. If the database service is unreachable at collector startup, bindings will fail and raw readings will not carry usable sensor IDs.

7.2.4 Extend preprocessing configuration (the data quality contract)

Preprocessing is configured via `dt/utils/preprocessing_config.yml`. The YAML is organized as:

- `templates`: reusable defaults for a sensor type/metric
- `sensors`: concrete keys that map a sensor (as stored in DB) to a template and/or overrides

To add a sensor, you typically:

1. add a new template (or reuse an existing one) describing validation, imputation, calibration, normalization, and units
2. add a `sensors` entry whose key matches the sensor's `SensorDescriptor.name` (exact match), or matches the derived generic `<sensor_type>.<metric>` key used as a fallback

```
1  templates:
2    co2.ppm:
3      units: "ppm"
4      validation:
5        range: { min: 0, max: 10000 }
6        roc: { max_per_minute: 500.0 }
7        stuck: { max_flat_seconds: 600 }
8      imputation:
9        strategy: window_average
10     window_seconds: 300
11     min_samples: 3
12     max_gap_seconds: 600
13     decay_seconds: 0
14     calibration:
15       strategy: identity
16     normalization:
17       strategy: min_max
18       input_min: 0.0
19       input_max: 10000.0
20       output_min: 0.0
21       output_max: 1.0
22       clip: true
23
24  sensors:
25    co2.ppm: { template: co2.ppm }
26    # or a specific override key matching the DB sensor name:
27    # sensors.basil.co2.001.ppm: { template: co2.ppm, validation: { range: { min: 350,
28                                   max: 2000 } } }
```

YAML

Internally, `ConfigurationManager` (`dt/data/preprocess/config/manager.py`) builds a sensor registry by calling the database service (`GET /sensors`) and mapping each `sensor_id` to a YAML key. If the registry cannot be built, preprocessing will not know how to configure that sensor stream.

7.2.5 Extend the dashboard subscriptions and charts (the UX contract)

The webapp backend already subscribes to all processed sensor topics discovered through `Topics.list_sensor_topics()` and forwards them to `Socket.IO` using the processed topic string as the `Socket.IO` event name.

The frontend still needs to know which processed topics exist and how to visualize them. Update:

- `dt/webapp/frontend/src/views/analytics/realtime_topics.ts` to add the new processed topic
- `dt/webapp/frontend/src/views/analytics/realtime_readings_store.ts` to subscribe to it
- `dt/webapp/frontend/src/views/analytics/realtime_monitoring_model.ts` to add a chart title and y-axis configuration

If you forget this step, the backend will emit events, but the UI will not subscribe and the new stream will appear “silent”.

7.3 Adding an alert rule

The intended workflow is: keep the evaluator logic generic, and express most alert behaviour in YAML. If you find yourself wanting to add custom Python “if/else” for a single plant or sensor, stop and consider whether the YAML schema should be extended instead.

7.3.1 Rule matching: what does `source` mean?

The alert evaluator extracts `source` from the processed payload as `payload.topic.short_name`. Concretely:

- topic `dt.sensors.processed.temperature` → `source == "temperature"`
- topic `dt.sensors.processed.soil_moisture` → `source == "soil_moisture"`

Your rule’s `source` must match that short name, or be `"*"` to apply to all sensors.

7.3.2 Rule definition structure

Rules live in `dt/utils/alert_rules.yml` under the `alert_rules:` key. Each rule defines:

- **identity:** `rule_id`, `name`
- **operator-facing message:** `description` (a Python `str.format(...)` template)
- **routing:** `evaluation_stage` (currently only `processed`) and `source`
- **condition:** one of `threshold`, `range`, `dq_score`, `validation_flag`
- **noise controls:** `persistence_count` (consecutive violations) and `cooldown_seconds`

Example (threshold):

```
1 - rule_id: temp_high_warning
2   name: High Temperature Warning
3   description: "Temperature exceeds {threshold}°C (value={value})"
4   severity: warning
5   evaluation_stage: processed
6   source: temperature
7   condition:
8     type: threshold
```

YAML


```

9     operator: ">"
10    threshold: 35.0
11    persistence_count: 2
12    cooldown_seconds: 300

```

The `description` template is formatted with a merged context that includes both:

- payload fields (such as `{value}`, `{unit}`, `{dq_score}`, `{plant_id}`, `{sensor_id}`)
- condition parameters (such as `{threshold}`, `{min_value}`, `{max_value}`)

Keeping the description template expressive is useful operationally: it reduces the need to cross-reference raw readings when triaging an alert.

7.3.3 Persistence and cooldown (how to avoid alert spam)

The in-memory registry tracks occurrences and implements two suppression mechanisms:

- `persistence_count`: the rule must be violated this many times before an alert becomes publishable
- `cooldown_seconds`: once an alert is active, repeated violations are suppressed until cooldown expires, after which the alert can be re-published as an update

Tune these values with the sensor's sampling rate in mind. A persistence count of 5 means "5 consecutive micro-batches that triggered the rule", which can translate to minutes depending on how often readings arrive.

7.3.4 Validation strategy (prove the rule works)

Whenever possible, validate a new rule against real or replayed historical processed readings. The minimum bar is:

1. confirm `source` matching by checking the topic short name used in the processed stream
2. trigger the condition with a controlled input (or a small replay)
3. verify the rule is suppressed below persistence threshold and inside cooldown
4. verify the published alert contains the expected message fields and severity

7.4 Extending preprocessing

Preprocessing is intentionally split into two extension surfaces:

1) **processors** (pipeline stages) which are responsible for a single concern and update the `ProcessingContext`, and 2) **strategies** (per-sensor algorithms) selected by configuration and instantiated by factories.

7.4.1 Extending an existing stage by adding a strategy

If you want to add an alternative algorithm (for example, a new smoothing method), prefer adding a strategy rather than adding branching logic in a processor. The typical steps are:

1. Add a new typed config dataclass in `dt/data/preprocess/config/types.py` with a distinct `strategy` literal string.
2. Ensure the config loader can deserialize it (see the configuration serialization helpers under `dt/data/preprocess/config/`).
3. Implement the strategy in the stage's `strategies.py` and register it in the corresponding `build_*_strategy(...)` factory.
4. Add focused unit tests for the strategy behaviour (pure Python, no Spark).

5. Update `dt/utils/preprocessing_config.yml` to reference the new strategy for a template or sensor override.

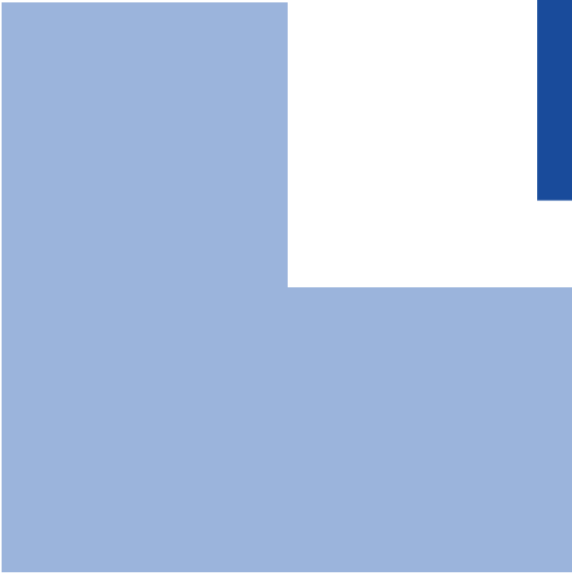
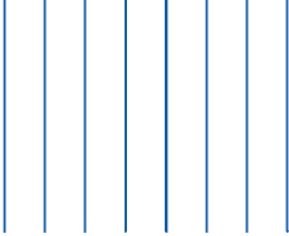
Because strategies receive a `StateProvider`, they can be stateful without leaking Spark details into the algorithm. The `SparkStateProvider` is responsible for translating Spark's `GroupState` into the typed `SensorState` model.

7.4.2 Adding a new stage (processor) to the pipeline

Adding a new stage is a bigger change because it alters the pipeline shape. Only do this when the existing stages cannot reasonably express the new behaviour. The minimal design rules are:

- create a new processor under `dt/data/preprocess/stages/<stage>/processor.py` implementing `BaseProcessor`
- store intermediate results on `ProcessingContext` (and keep names domain-focused)
- keep the processor deterministic and side-effect free (no I/O, no network calls)
- wire the stage into `PipelineBuilder.build_standard_pipeline()` in `dt/data/preprocess/core/pipeline.py`

If the stage needs configuration, extend the YAML schema and typed config in one coherent change. If the stage needs to surface new fields downstream, you are changing a shared contract: you will need to update `ProcessedSensorData` and follow through on adapters, storage persistence, and UI payload shaping. Treat that as a coordinated change, not a local refactor.



ULB